

# Squid Ink-Pulse

## *Proyecto Integrador Programación Avanzada*

### **Nombre del equipo:**

- Yeco Works

### **Integrantes:**

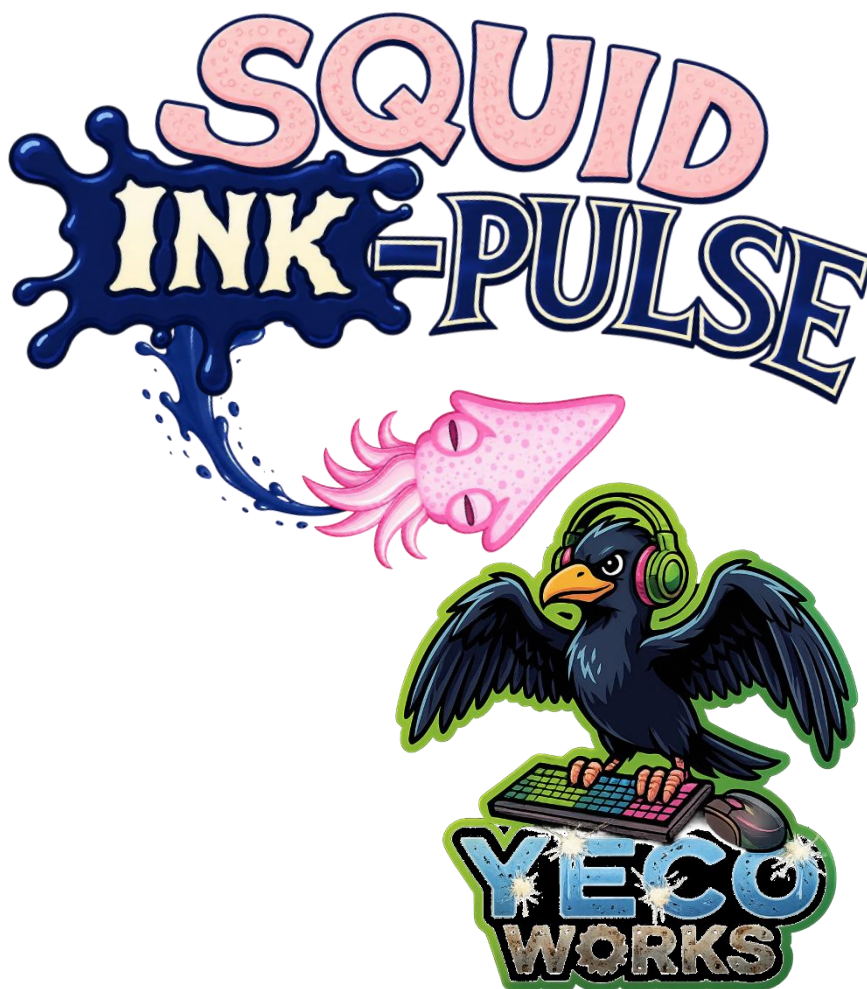
- Inti Santibáñez (ICCI)
- Mauricio Muñoz (ICCI)
- Matías Palacios (ITI)
- Pablo Guzmán (ICCI)
- Rodrigo Cortés (ICCI)

### **Profesor:**

- Bastián Braulio Ruiz Garay

### **Fecha de Entrega:**

- 14 de Junio del 2026
- \*Preentrega: 10 de Junio



**Coquimbo, 2026**

# Índice

|                                           |           |
|-------------------------------------------|-----------|
| <b>NOTAS DE ACTUALIZACIÓN</b>             | <b>3</b>  |
| <b>GLOSARIO</b>                           | <b>5</b>  |
| <b>RESUMEN EJECUTIVO</b>                  | <b>7</b>  |
| <b>FICHA DE DESARROLLO</b>                | <b>8</b>  |
| INFORMACIÓN GENERAL                       | 8         |
| ESPECIFICACIONES                          | 9         |
| <b>DESCRIPCIÓN GENERAL DEL EQUIPO</b>     | <b>10</b> |
| TABLA DE INTEGRANTES                      | 10        |
| COMPROMISOS DEL EQUIPO                    | 10        |
| <b>DESCRIPCIÓN GENERAL DEL VIDEOJUEGO</b> | <b>11</b> |
| DISEÑO Y ESTILO                           | 11        |
| NARRATIVA Y CONTEXTO                      | 11        |
| <i>Secuestro Percibido:</i>               | 11        |
| <i>Plot-Twist y Loop Narrativo</i>        | 11        |
| AMBIENTACIÓN                              | 11        |
| <i>Mundo Submarino</i>                    | 11        |
| <i>Amenazas</i>                           | 12        |
| PERSONAJES                                | 13        |
| <i>Protagonista: Baby Squid</i>           | 13        |
| <i>Madre: Mommy Squid</i>                 | 13        |
| <i>Enemigos y amenazas</i>                | 13        |
| <i>Camarones</i>                          | 14        |
| <i>Gadgets</i>                            | 14        |
| <i>Pez dealer</i>                         | 14        |
| PÚBLICO OBJETIVO                          | 15        |
| <i>Clasificación</i>                      | 15        |
| REFERENCIAS                               | 15        |
| <b>JUGABILIDAD</b>                        | <b>16</b> |
| OBJETIVO DEL JUGADOR                      | 16        |
| <i>Objetivo Principal</i>                 | 16        |
| <i>Objetivo Identitario</i>               | 16        |
| <i>Objetivos Complementarios</i>          | 16        |
| MECÁNICAS                                 | 17        |
| <i>Principal</i>                          | 17        |
| <i>Diferenciadora</i>                     | 18        |
| <i>Time based Boss</i>                    | 19        |
| <i>Portales</i>                           | 20        |
| <i>Gadgets y desbloqueables</i>           | 21        |
| <i>Tienda (Meta Game)</i>                 | 21        |
| <i>Tienda (In-Run)</i>                    | 22        |
| <i>Menú de opciones</i>                   | 22        |
| ENEMIGOS Y OBSTÁCULOS                     | 22        |
| <i>Tabla de Enemigos</i>                  | 22        |
| <i>SS Carnage (Barco humano pesquero)</i> | 23        |
| GADGETS                                   | 24        |

|                                                           |           |
|-----------------------------------------------------------|-----------|
| <i>Tabla de gadgets</i> .....                             | 24        |
| ESTADOS DEL JUEGO .....                                   | 24        |
| <i>Condiciones</i> .....                                  | 24        |
| CICLO DE JUEGO .....                                      | 25        |
| DIAGRAMA DE FLUJO .....                                   | 25        |
| <b>SISTEMA DE PROGRESIÓN</b> .....                        | <b>26</b> |
| PROGRESIÓN INTERNA (IN RUN) .....                         | 26        |
| PROGRESIÓN EXTERNA (METAGAMING) .....                     | 26        |
| CONTROLES.....                                            | 26        |
| RETENCIÓN DEL JUGADOR .....                               | 27        |
| <b>ESTADO DE CUMPLIMIENTO</b> .....                       | <b>28</b> |
| RESUMEN DE CUMPLIMIENTO POR COMPONENTE .....              | 28        |
| VISUALIZACIÓN DE IMPLEMENTACIONES .....                   | 29        |
| PENDIENTES PRINCIPALES.....                               | 30        |
| <b>DESARROLLO</b> .....                                   | <b>31</b> |
| PROGRAMACIÓN .....                                        | 32        |
| ARQUITECTURA.....                                         | 33        |
| <i>Persistencia</i> .....                                 | 34        |
| DOCUMENTACIÓN .....                                       | 34        |
| TESTING .....                                             | 35        |
| <b>ANÁLISIS DE COSTOS</b> .....                           | <b>37</b> |
| ESCENARIOS CONSIDERADOS .....                             | 37        |
| RESUMEN COMPARATIVO DE COSTOS.....                        | 37        |
| <i>Elementos considerados en ambos escenarios</i> .....   | 38        |
| <i>Criterios utilizados para la estimación</i> .....      | 38        |
| <i>Desglose de costos: MVP</i> .....                      | 39        |
| <i>Elementos no considerados en el MVP</i> .....          | 40        |
| <i>Posibles fondos o vías de financiamiento</i> .....     | 42        |
| <b>METODOLOGÍA DE TRABAJO Y PLANIFICACIÓN</b> .....       | <b>43</b> |
| CARTA GANTT - ACTUALIZADA.....                            | 43        |
| <i>Hitos</i> .....                                        | 43        |
| <i>Áreas y detalle</i> .....                              | 44        |
| HERRAMIENTAS DE TRABAJO .....                             | 45        |
| SPRINTS EJECUTADOS / BITÁCORA .....                       | 46        |
| <i>Sprint 1 — S1: Definiciones</i> .....                  | 46        |
| <i>Sprint 2 — S2: Base Visual y Técnica</i> .....         | 47        |
| <i>Sprint 3 — S3: Prototipo Jugable</i> .....             | 48        |
| <i>Sprint 4 — S4: Mecánicas clave MVP</i> .....           | 49        |
| <i>Sprint 6 — S6: Presentación y segunda escena</i> ..... | 51        |
| <i>Sprint 7 — S7: Últimas implementaciones</i> .....      | 52        |
| <b>CONCLUSIONES</b> .....                                 | <b>53</b> |
| ALCANCE ACTUAL DEL PROYECTO .....                         | 53        |
| VIABILIDAD DEL PROYECTO.....                              | 53        |
| AVANCE E IMPLEMENTACIÓN .....                             | 53        |
| APOORTE DE SCRUM AL DESARROLLO .....                      | 53        |
| CIERRE GENERAL.....                                       | 53        |
| <b>REFERENCIAS Y BIBLIOGRAFÍA</b> .....                   | <b>54</b> |

## Notas de actualización

El presente informe corresponde a una actualización del primer informe de Squid Ink-Pulse. La estructura general del documento se mantiene en sus apartados principales, pero se incorporan ajustes para reflejar con mayor precisión el estado actual del MVP, el avance técnico del proyecto y las tareas pendientes posteriores a la segunda entrega.

### Títulos y subtítulos mantenidos

Se mantienen los apartados base del primer informe: Ficha de desarrollo, Descripción general del equipo, Descripción general del videojuego, Jugabilidad, Sistema de progresión, Metodología de trabajo y planificación, Herramientas de trabajo, Sprints ejecutados / bitácora, Conclusión y Referencias bibliográficas.

### Títulos y subtítulos agregados

Se incorporan nuevos apartados orientados a evidenciar el avance real del proyecto:

- Notas de actualización.
- Estado de cumplimiento.
  - Visualización.
  - Resumen de cumplimiento por componente.
  - Pendientes principales.
- Desarrollo.
  - Programación.
  - Arquitectura.
  - Persistencia.
  - Documentación.
  - Testing.
- Análisis de costos.
  - Escenarios considerados.
  - Resumen comparativo de costos.
  - Desglose de costos: MVP.
  - Elementos no considerados en el MVP.
  - Posibles fondos o vías de financiamiento.
- Carta Gantt actualizada.
- Sprints posteriores a la primera entrega.
- Alcance actual del proyecto.

## **Títulos y subtítulos actualizados**

Se actualizaron apartados ya existentes para ajustarlos al estado real del proyecto:

- Resumen Ejecutivo: fue reformulado para reflejar no solo la propuesta conceptual del juego, sino también el estado actual del MVP, sus sistemas implementados, sus pendientes principales y su organización técnica.
- Glosario adicionó nuevos términos y sus definiciones en el contexto.
- Conclusiones: fueron reemplazadas por un cierre más preciso, organizado en torno al alcance actual, la viabilidad del proyecto, lo ya implementado y el aporte de SCRUM al avance del desarrollo.
- Diagrama de flujo fue actualizado por uno más visible y coherente.
- Carta Gantt: fue actualizada para distinguir hitos cumplidos, tareas en proceso y actividades pendientes.
- Sprints ejecutados / bitácora: se amplió para incorporar los sprints posteriores, incluyendo avances en mecánicas clave, tienda, portales, segunda escena, presentación, spawn, tutorial y ajustes finales.
- Jugabilidad y Sistema de progresión: fueron ajustados para representar mejor el estado real del MVP y no solo su diseño proyectado.
- Referencias para las fuentes de los fondos citados y los costos de sueldos referencia.

## **Títulos y subtítulos eliminados o absorbidos**

Algunos apartados del primer informe fueron eliminados como secciones independientes o absorbidos por apartados más técnicos:

- Viabilidad y alcance del proyecto.
  - MVP.
  - Análisis de riesgos.
  - Estrategias de mitigación.
  - Extensibilidad.
- Mensaje trascendente.
  - Para el jugador.
  - Para la sociedad.
- Valor del proyecto.
- Proyección.

Estos contenidos no desaparecen por completo, sino que se integran de forma más acotada en secciones como Estado de cumplimiento, Desarrollo, Análisis de costos, Pendientes principales y Conclusiones.

## Glosario

**Endless Runner:** Subgénero de videojuego caracterizado por el desplazamiento continuo del personaje, donde el objetivo es sobrevivir el mayor tiempo posible o recorrer la mayor distancia frente a una dificultad creciente.

**Gameplay Loop (Ciclo de juego):** Secuencia recurrente de acciones que define la experiencia del jugador. En este proyecto: avanzar, esquivar, asumir riesgos, cargar Ink-Pulse, usarlo en momentos críticos y repetir.

**Game Over:** Estado que marca el fin de una partida cuando el jugador pierde.

**Side-scrolling:** Desplazamiento lateral continuo del escenario o del personaje, característico de juegos 2D donde el avance principal ocurre en una dirección horizontal.

**Singleplayer:** Modalidad de juego para un solo jugador.

**Roguelite:** Enfoque de diseño basado en partidas repetibles con reinicio frecuente y algún grado de progresión entre intentos.

**Cooldown (tiempo de reutilización):** Intervalo mínimo que debe transcurrir antes de que un evento, habilidad o sistema pueda volver a activarse.

**Run:** Una partida individual del juego, desde su inicio hasta el game over.

**In Run:** En medio de una partida.

**Out of Run:** Fuera de una partida.

**Skills Upgrade:** Mejora de las características fijas del personaje.

**Dash:** Desplazamiento rápido y breve que permite evadir peligros o superar eventos críticos.

**Ink-Pulse:** Mecánica central del juego que permite ejecutar un impulso (dash) tras cargar una barra mediante la proximidad controlada a amenazas. Constituye el eje del sistema de riesgo-recompensa.

**Graze / Graze Zone / Zona de proximidad:** Interacción en la que el jugador se aproxima a un obstáculo o enemigo sin colisionar. En este contexto, dicha proximidad permite la recarga del Ink-Pulse.

**Time-Based:** Evento estructurado que tiene cierta durabilidad, basado en el tiempo.

**Progresión in-run:** Evolución de la dificultad y de las condiciones de juego dentro de una misma partida, mediante variables como velocidad, densidad de enemigos o aparición de eventos.

**Metaprogresión / Metagame / Metagaming (progresión externa):** Sistema de avance persistente entre partidas, basado en la acumulación de recursos y desbloqueo de mejoras permanentes.

**Gadget:** Recurso de uso situacional que otorga ventajas temporales durante la partida, introduciendo decisiones tácticas en tiempo real.

**Gadget pasivo:** Tipo de gadget que se activa automáticamente bajo condiciones específicas, sin intervención directa del jugador.

**Gadget activo:** Tipo de gadget que requiere activación manual mediante controles asignados, implicando toma de decisiones en tiempo real.

**Hitbox (zona de colisión):** Área definida de un objeto o personaje que determina la detección de impactos dentro del sistema de juego.

**Spawn:** Proceso de aparición de enemigos, obstáculos o eventos dentro del entorno de juego, regido por reglas de generación.

**SCRUM:** Metodología ágil de gestión de proyectos basada en iteraciones cortas (sprints), planificación continua y revisión periódica del progreso.

**UI (User Interface / Interfaz de usuario):** Conjunto de elementos visuales con los que interactúa el jugador, como menús, indicadores, botones y barras.

**Slot:** Espacio disponible dentro de un inventario para almacenar un objeto o recurso.

**Pacing (ritmo de juego):** Forma en que el juego distribuye tensión, descanso, intensidad y eventos a lo largo de la experiencia.

**Zoom out:** Alejamiento de la cámara para ampliar el campo visible y mejorar la lectura de la escena.

**Sprite:** Recurso gráfico bidimensional utilizado para representar personajes, objetos, efectos o elementos de interfaz dentro del juego.

**QA (Quality Assurance / Aseguramiento de calidad):** Conjunto de tareas orientadas a verificar que el producto cumpla estándares funcionales, técnicos y de experiencia de usuario.

**Testing:** Proceso de prueba sistemática del juego para detectar errores, evaluar balance y validar el funcionamiento de las mecánicas.

**Build:** Versión compilada y ejecutable de un software o videojuego, generada a partir del código fuente en un momento determinado del desarrollo.

**Boss:** Enemigo o evento especial de mayor complejidad que interrumpe el flujo habitual del juego y exige una respuesta distinta por parte del jugador.

**MVP (Minimum Viable Product / Producto Mínimo Viable):** Versión mínima funcional de un proyecto que permite validar su propuesta central con el menor alcance posible.

## Resumen Ejecutivo

El presente informe expone el diseño, desarrollo y estado actual de Squid Ink-Pulse, un videojuego endless runner 2D desarrollado en Unity por el equipo Yeco Works. La propuesta se basa en una experiencia de avance continuo, evasión de amenazas y toma de decisiones rápidas, incorporando como mecánica diferenciadora el Ink-Pulse: un impulso que se carga al aproximarse al peligro sin colisionar.

Desde el punto de vista narrativo, el juego sigue a Baby Squid, un calamar bebé que persigue a su madre tras una aparente captura en un entorno submarino hostil. Esta premisa se integra con el ciclo de reintento del género, ya que cada derrota se vincula con el rescate del protagonista por parte de su madre, reforzando una idea de aprendizaje, crecimiento y adaptación.

Actualmente, el proyecto se encuentra en una etapa de MVP funcional en cumplimiento parcial. Ya se han implementado sistemas centrales como movimiento, Ink-Pulse, detección de proximidad, enemigos, camarones, gadgets, tienda in-run, portales y evento de boss. Sin embargo, aún quedan pendientes relevantes, entre ellos persistencia completa, tienda global, tutorial, balance, menú de opciones, QA formal y pulido audiovisual.

A nivel técnico y metodológico, el desarrollo se organiza mediante una arquitectura modular por dominios, apoyada en Unity, C#, GitHub, Jira y SCRUM. El informe también incorpora una actualización de planificación, bitácora de sprints, análisis de costos y posibles vías de financiamiento. En conjunto, Squid Ink-Pulse se presenta como una propuesta coherente, viable y con potencial de expansión, aunque todavía requiere ajustes finales para consolidar una build más estable.



# Ficha de Desarrollo

## Información general

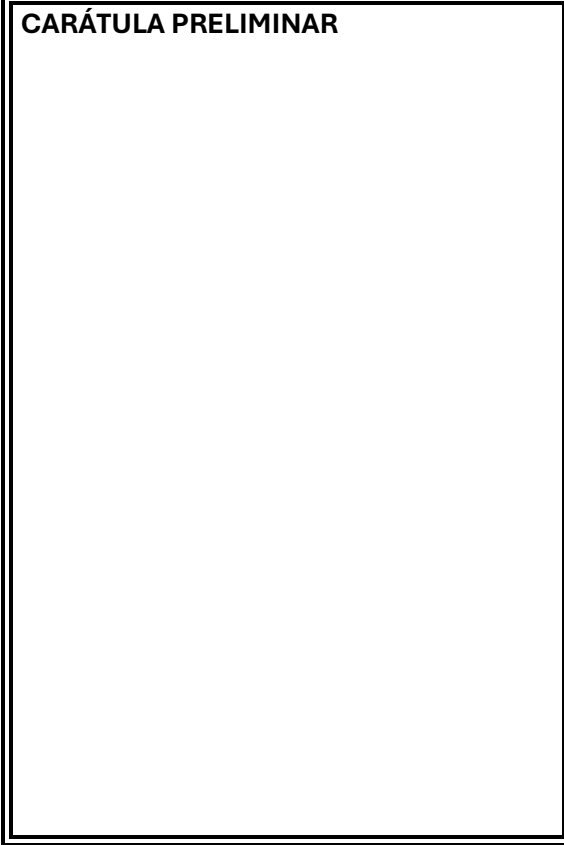
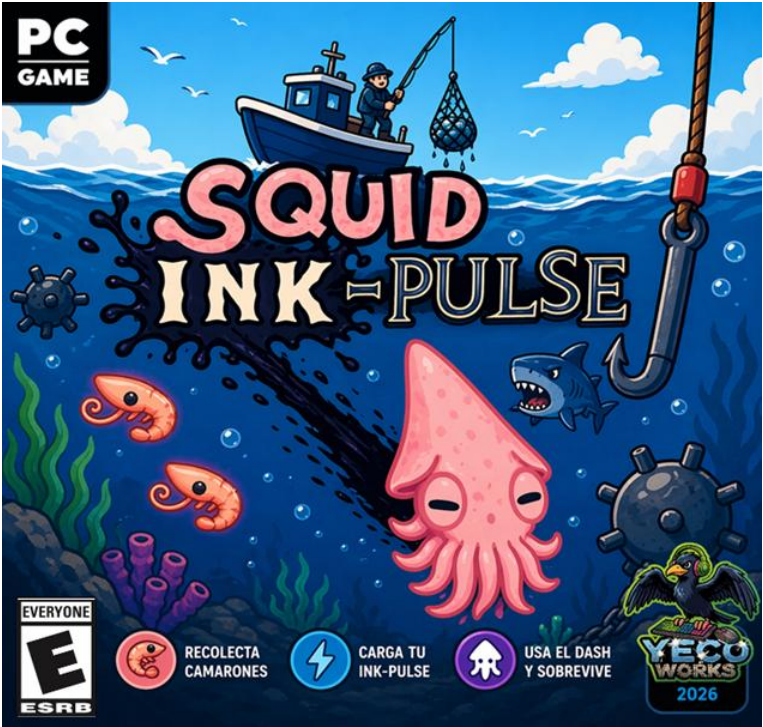
|                       |                                                                                                                                                                                                                                                                 |
|-----------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| NOMBRE DEL JUEGO      | Squid Ink-Pulse                                                                                                                                                                                                                                                 |
| EQUIPO DE DESARROLLO  | Yeco Works                                                                                                                                                                                                                                                      |
| GÉNERO                | Endless Runner                                                                                                                                                                                                                                                  |
| ESTILO                | 2D lateral con side-scrolling                                                                                                                                                                                                                                   |
| PLATAFORMA OBJETIVO   | PC                                                                                                                                                                                                                                                              |
| MOTOR DE JUEGO        | Unity                                                                                                                                                                                                                                                           |
| LENGUAJE              | C#                                                                                                                                                                                                                                                              |
| MODALIDAD             | Singleplayer                                                                                                                                                                                                                                                    |
| CLASIFICACIÓN         | E                                                                                                                                                                                                                                                               |
| METODOLOGÍA           | SCRUM de Sprints semanales                                                                                                                                                                                                                                      |
| HERRAMIENTAS DE APOYO | Unity, GitHub, Jira, Canva y Discord                                                                                                                                                                                                                            |
| CARÁTULA PRELIMINAR   | <div></div> <div><p>Figura 1. Portada de Squid Ink-Pulse. Generado con ChatGPT</p></div> |

Tabla 1. Información General de Squid Ink Pulse. Elaboración Propia.

## Especificaciones

|                                |                                                                                                                                                |
|--------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>PÚBLICO OBJETIVO</b>        | Jugadores casuales orientados a reflejos y precisión<br>Jugadores competitivos que buscan romper récords                                       |
| <b>PREMISA</b>                 | Un calamar bebé persigue a su madre en un entorno submarino hostil, enfrentando peligros crecientes.                                           |
| <b>OBJETIVO DEL JUGADOR</b>    | Sobrevivir el mayor tiempo posible evitando obstáculos y optimizando el uso de habilidades.                                                    |
| <b>MECÁNICA PRINCIPAL</b>      | Movimiento continuo con esquivas de obstáculos.                                                                                                |
| <b>MECÁNICA DIFERENCIADORA</b> | “Ink-Pulse”: dash que se carga al asumir riesgos (proximidad a peligros).                                                                      |
| <b>CONDICIÓN DE DERROTA</b>    | Colisión con obstáculos o fallo en uso obligatorio del dash.                                                                                   |
| <b>RECURSOS DEL JUEGO</b>      | Camarones como moneda para mejoras.                                                                                                            |
| <b>SISTEMA DE PROGRESIÓN</b>   | In Run: Incremento de dificultad por tiempo/distancia + mejoras mediante tienda in-game.<br>Out Run: Skills upgrade, bonificaciones generales. |
| <b>ENEMIGOS/OBSTÁCULOS</b>     | Entidades con comportamiento lineal, dinámico (ultimate de boss) y estático.                                                                   |
| <b>CICLO DE JUEGO</b>          | Evadir → Arriesgar → Cargar Ink-Pulse → Usar → Repetir                                                                                         |
| <b>CONTROLES</b>               | Mouse/teclado (movimiento, dash y gadgets)                                                                                                     |

**Tabla 2. Tabla de Especificaciones de Squid Ink-Pulse. Elaboración Propia.**

## Descripción general del Equipo

Yeco Works es un equipo de desarrollo indie basado en la metodología SCRUM o ágil.

El equipo Yeco Works está conformado por cinco integrantes del área de la informática, organizados bajo una estructura de trabajo basada en la metodología ágil SCRUM. Esta metodología permite desarrollar el proyecto de manera iterativa, con una distribución clara de responsabilidades y una comunicación constante entre los miembros.

Cada integrante cumple un rol específico dentro del equipo



**Figura 2. Logo de empresa.**  
**Elaboración propia.**

## Tabla de Integrantes

| INTEGRANTE             | ROL                     | TAREAS                                                                                           | CARRERA |
|------------------------|-------------------------|--------------------------------------------------------------------------------------------------|---------|
| <b>INTI SANTIBÁÑEZ</b> | QA / Tester             | Realiza pruebas y verifica la calidad del producto.                                              | ICCI    |
| <b>MAURICIO MUÑOZ</b>  | Gameplay Programmer     | Desarrolla las mecánicas y la lógica del juego, se enfoca en implementar las mecánicas de juego. | ICCI    |
| <b>MATÍAS PALACIOS</b> | Visual & Sound Designer | Diseña e implementa los elementos visuales y sonoros.                                            | ITI     |
| <b>PABLO GUZMÁN</b>    | SCRUM Master            | Coordina el equipo y asegura el cumplimiento de la metodología y los tiempos.                    | ICCI    |
| <b>RODRIGO CORTÉS</b>  | Product Owner           | Define los objetivos del producto y prioriza las tareas del desarrollo.                          | ICCI    |

**Tabla 3. Tabla de Integrantes. Elaboración Propia.**

- **ICCI: Ingeniería Civil en Computación e Informática.**
- **ITI: Ingeniería en Tecnologías de la Información.**

El equipo trabaja de forma colaborativa, manteniendo comunicación constante y cumpliendo con los compromisos definidos, como respetar plazos, revisar entregas y compartir avances de manera clara.

## Compromisos del equipo

1. Cumplir con los horarios establecidos para las reuniones y entregar los trabajos en las fechas acordadas, demostrando respeto por el tiempo del equipo.
2. Mantener un diálogo abierto y constante, compartiendo avances, inquietudes y dificultades de forma clara y oportuna.
3. Garantizar que cada entrega cumpla con los estándares esperados, revisando cuidadosamente el trabajo antes de presentarlo.

## Descripción general del videojuego

Squid Ink-Pulse es un endless runner 2D donde controlas a un calamar bebé que persigue a su madre tras su aparente captura. El jugador avanza constantemente esquivando enemigos y obstáculos en un entorno submarino cada vez más peligroso. Su rasgo distintivo es el Ink-Pulse: una habilidad que solo se recarga al pasar cerca del peligro, obligando a jugar de forma arriesgada. La dificultad aumenta progresivamente e introduce situaciones que exigen usar esta mecánica, generando un ciclo dinámico de riesgo, reacción y mejora continua.

## Diseño y estilo

El juego adopta un estilo visual cartoon, caracterizado por formas simples, colores saturados y alto contraste, lo que favorece la legibilidad en pantalla y la rápida identificación de amenazas. Este enfoque prioriza la claridad por sobre el detalle, coherente con un endless runner donde la toma de decisiones es inmediata. No se incluyen representaciones explícitas de violencia (sangre o daño gráfico); sin embargo, existe una sensación constante de hostilidad dada por el entorno y las amenazas, transmitida mediante animaciones, ritmo y composición visual, sin comprometer la accesibilidad.

## Narrativa y contexto

### Secuestro Percibido:

La narrativa se construye a partir de un conflicto inicial aparente: el protagonista, un calamar bebé, presencia cómo su madre es capturada por un pescador y reacciona instintivamente persiguiéndola. Esta premisa activa el juego y justifica el desplazamiento constante, dotando de sentido a la urgencia del avance y a la toma de riesgos por parte del jugador.

### Plot-Twist y Loop Narrativo

Sin embargo, el cierre de cada partida introduce un quiebre en la interpretación de los hechos: al perder, el calamar queda inconsciente y es rescatado por su madre, lo que sugiere que el “secuestro” no era tal, o al menos no en los términos en que el protagonista lo percibe. Esta situación configura un loop narrativo en el cual la madre le permite continuar, con el propósito de acompañar su proceso de crecimiento y evolución, marcando el tránsito de paralarva a calamar. Dado que el protagonista no comprende plenamente lo ocurrido, reinicia su acción bajo la misma premisa inicial, reforzando así la coherencia entre la narrativa y la lógica de reintento propia del género endless runner.

## Ambientación

### Mundo Submarino

La ambientación se sitúa en un mundo submarino dinámico, caracterizado por variaciones de profundidad, iluminación y densidad de elementos en pantalla, lo que aporta diversidad visual y refuerza la progresión del juego. Este entorno no es meramente decorativo: condiciona la jugabilidad mediante obstáculos naturales y artificiales que emergen de forma continua.

## Zona Epipelágica

Corresponde al nivel inicial del juego, ubicado en las capas más superficiales del océano. Se caracteriza por una alta visibilidad, colores más vivos y menor densidad de amenazas, facilitando la adaptación del jugador al ritmo y controles. Desde el punto de vista de diseño, funciona como una zona de introducción progresiva, donde se presentan las mecánicas base. A medida que avanza la partida, el jugador se ve forzado a cambiar de profundidad debido a la presión de los enemigos y obstáculos, integrando la verticalidad como elemento de decisión.



**Figura 3. Fondo Zona Epipelágica.**  
**Elaboración Propia.**

## Zona Abisopelágica

Es la segunda zona considerada para el MVP y representa un salto en complejidad y atmósfera. Predominan la oscuridad, la iluminación puntual y una mayor sensación de peligro. Este cambio no es solo estético: condiciona la jugabilidad al reducir la anticipación visual y aumentar la dependencia de reflejos y memoria de patrones. Además, permite enriquecer la experiencia mediante efectos de luz y contraste, reforzando la tensión del entorno.

## Amenazas

La ambientación del juego no solo define el aspecto visual, sino también la naturaleza de los desafíos que enfrenta el jugador. En este sentido, las amenazas se estructuran en tres categorías complementarias que aportan coherencia al mundo y variedad a la jugabilidad

- **Amenaza Humana**

Las actividades humanas asociadas a la pesca se presentan como el principal agente externo de peligro. Elementos como ganchos, redes, residuos y artefactos irrumpen en el ecosistema marino introduciendo una lógica ajena y agresiva. Estas amenazas funcionan como obstáculos críticos y enemigos indirectos, estableciendo una tensión constante entre el entorno natural y la intervención humana, lo que además aporta sentido al contexto narrativo.

- **Amenazas Submarinas**

El ecosistema marino en sí mismo es hostil. La presencia de depredadores y condiciones adversas configura un entorno donde la supervivencia depende de la evasión, la reacción y la adaptación continua. Estas amenazas representan la lógica natural del mundo del juego, reforzando la vulnerabilidad del protagonista y evitando que la dificultad se perciba como arbitraria.

- **Amenazas de Entorno**

A lo anterior se suman elementos propios del escenario, como derrumbes, géiseres submarinos, formaciones rocosas o estructuras que bloquean el paso. Estas amenazas introducen restricciones espaciales y variabilidad en el recorrido, obligando al jugador a modificar su trayectoria y tomar decisiones rápidas en función del entorno inmediato.

En conjunto, estas tres dimensiones configuran un sistema de amenazas coherente, donde cada tipo cumple un rol específico en la experiencia: presión externa (humana), supervivencia natural (biológica) y condicionamiento del espacio (ambiental).

## Personajes

### Protagonista: Baby Squid

Es un calamar bebé que actúa impulsado por una reacción instintiva más que racional. Su comportamiento refleja urgencia y vulnerabilidad, lo que se traduce en una jugabilidad centrada en reflejos y toma de riesgos. Mecánicamente, es el eje del sistema: su movilidad constante y la gestión del Ink-Pulse definen la experiencia del jugador.



**Figura 4. Baby Squid.**  
**Elaboración Propia.**

### Madre: Mommy Squid

Cumple un rol principalmente narrativo. Es el detonante del conflicto inicial y, a la vez, quien cierra el ciclo en cada derrota al rescatar al protagonista. Su presencia refuerza el loop narrativo y aporta coherencia al sistema de reintento, sin intervenir directamente en la jugabilidad.



**Figura 5. Boceto de Mommy Squid.**  
**Elaboración Propia.**

## Enemigos y amenazas

### Comunes

Conforman el núcleo del desafío y se dividen en distintos tipos según su comportamiento:

- **Peces globo:** obstáculos móviles que ocupan espacio y limitan rutas de escape.
- **Cañas de pesca y anzuelos:** amenazas externas que irrumpen desde fuera del entorno natural, con trayectorias variables.
- **Minas submarinas:** elementos estáticos que castigan la falta de precisión.

### Enemigos Especiales

Los enemigos especiales cumplen una función estructural dentro del diseño: forzar el uso del Ink-Pulse en momentos críticos. A diferencia de los obstáculos convencionales, no están pensados para ser esquivados mediante habilidad básica, sino para introducir una condición obligatoria de decisión, donde el jugador debe haber gestionado correctamente su recurso de adrenalina.

- **SS Carnage:** Es la manifestación principal de esta lógica. Representa la embarcación asociada al supuesto secuestro y aparece como un evento de alta presión que altera el flujo normal de la partida. Su mecánica central consiste en generar una “pared” o situación ineludible, mediante un ataque masivo y cerrando el espacio navegable que solo puede superarse utilizando el Ink-Pulse.

Desde el punto de vista de diseño, el SS Carnage:

- Valida la mecánica principal, al exigir el uso efectivo del Ink-Pulse.
- Penaliza la pasividad, ya que un jugador que no haya asumido riesgos previamente no dispondrá del recurso necesario.
- Introduce ritmo, funcionando como un punto de clímax dentro del ciclo de juego.

De este modo, no solo actúa como enemigo, sino como un mecanismo de control del comportamiento del jugador, asegurando que la experiencia se alinee con la identidad del juego basada en riesgo y recompensa.

En conjunto, estos elementos configuran un sistema de amenazas diverso que obliga a una adaptación constante y refuerza la identidad del juego basada en el riesgo.

## Camarones

Funcionan como la moneda principal del juego. Se obtienen durante la partida y permiten realizar compras tanto *in-run* (durante la ejecución) como fuera de ella, vinculando la experiencia inmediata con la progresión del jugador.



**Figura 6. Camarón.**  
**Elaboración Propia.**

## Gadgets

Son elementos almacenados en el inventario durante la partida que introducen variabilidad y toma de decisiones estratégicas. Su función es complementar las mecánicas base, ofreciendo ventajas situacionales y adaptabilidad frente a distintos escenarios.

### Pasivos

Se activan automáticamente bajo condiciones específicas, sin intervención directa del jugador. Están orientados a mitigar errores y extender la supervivencia.

- **Shell Shield:** Al recibir un impacto, se activa automáticamente generando una protección que evita la derrota inmediata. Funciona como un mecanismo de segunda oportunidad.  
**Condición:** solo puede aparecer una vez que el jugador supera los **5 minutos de run**, incentivando el progreso.

### Activos

Requieren activación manual mediante teclas asignadas (“Q”, “W”), introduciendo decisiones tácticas en tiempo real.

- **Ink Bottle:** Rellena instantáneamente la barra de adrenalina (Ink-Pulse), permitiendo responder a eventos críticos o preparar situaciones de riesgo.

## Pez dealer

Actúa como el intermediario entre el jugador y los gadgets durante la partida. Ofrece mejoras y recursos a cambio de camarones, incorporando una capa de decisión económica en tiempo real. Su presencia introduce pausas estratégicas pero limitadas dentro del flujo de juego, sin romper la dinámica general.



**Figura 7. Imagen de Realistic Fish Head,**  
**Utilizado en la presentación para**  
**representar al pez dealer.**  
**Propiedad de Nickelodeon (Bob Esponja)**

## Público Objetivo

El diseño del juego permite abarcar dos subgrupos bien definidos:

- **Jugadores casuales:** atraídos por la simplicidad de control y sesiones cortas, donde el desafío se basa en reflejos y adaptación progresiva.
- **Jugadores competitivos:** motivados por la superación de récords, optimización de rutas y dominio del sistema de riesgo asociado al *Ink-Pulse*.

## Clasificación

En coherencia con su contenido y enfoque, el juego se alinea con una clasificación E (Everyone), al presentar situaciones de tensión y peligro sin elementos gráficos explícitos. La ausencia de violencia directa, junto con su estilo visual accesible, lo posiciona como una experiencia apta para todo tipo de audiencias.



**Figura 8. E.**  
*Extraída de Wikipedia*

## Referencias

### Subway Surfers:

Referente directo en estructura de endless runner: avance automático, aumento progresivo de dificultad y énfasis en reflejos. Sirve como base para el ritmo de juego y la claridad en los objetivos inmediatos del jugador.

### Flappy Bird:

Aporta el enfoque de control simple pero exigente, donde pequeñas decisiones tienen consecuencias inmediatas. Influye en la precisión requerida y en la naturaleza punitiva del error.

### Deltarune (mecánica de graze):

Inspira la mecánica central de riesgo: la idea de obtener beneficios al acercarse al peligro sin colisionar. Este principio se traduce directamente en la recarga del Ink-Pulse, siendo clave en la identidad del juego.

### The Binding of Isaac:

Referencia para la estructura roguelite ligera, especialmente en la progresión mediante mejoras y reintentos constantes. Influye en el sistema de recompensas, la tienda y la rejugabilidad.



## Jugabilidad

A grandes rasgos, consiste en una experiencia de avance continuo en la que el jugador debe sobrevivir el mayor tiempo posible dentro de un entorno submarino dinámico y progresivamente más desafiante. A través de un control centrado en el posicionamiento y la evasión, se enfrenta a una serie de obstáculos y amenazas que exigen respuestas rápidas y precisas en tiempo real.

### Objetivo del jugador

#### Objetivo Principal

El objetivo principal es alcanzar la mayor distancia posible sin colisionar, manteniéndose con vida en un entorno que incrementa progresivamente su dificultad. Este propósito se traduce en una ejecución constante de decisiones rápidas, donde el jugador debe equilibrar evasión, posicionamiento y anticipación frente a amenazas dinámicas.

#### Objetivo Identitario

De forma complementaria, el jugador busca optimizar la gestión del Ink-Pulse, recargando la habilidad mediante la exposición controlada al peligro y utilizándola estratégicamente en eventos críticos. Este sistema redefine el objetivo tradicional del endless runner, ya que no basta con evitar riesgos, sino que es necesario asumirlos de manera calculada para sostener el progreso.

#### Objetivos Complementarios

Finalmente, existe un objetivo secundario de acumulación de recursos (camarones), y el desbloqueo de utilidades, que permiten acceder a mejoras y refuerza la progresión entre partidas, incentivando la repetición y el perfeccionamiento continuo del desempeño.

## Mecánicas

### Principal

El juego se estructura sobre un desplazamiento automático continuo. A esto se suma la recolección de recursos (camarones) y la adaptación constante a patrones de amenazas. La dificultad escala progresivamente en función del tiempo y la distancia recorrida, aumentando la densidad y complejidad de los desafíos.

#### ➤ Funcionalidad Técnica

El sistema de dificultad del juego no se modela como un crecimiento continuo simple, sino como una progresión segmentada en fases, donde variables clave se alternan y se reinician parcialmente ante eventos críticos.

#### Movimiento del jugador

El calamar se desplaza verticalmente en función de la posición del mouse, siguiendo un vector dependiente del eje y. Este enfoque permite un control fluido y preciso, centrado en la evasión y el posicionamiento.

La cámara sigue al jugador con libertad vertical, reforzando la sensación de exploración y evitando una experiencia rígida.

#### Velocidad del entorno

La velocidad del escenario sigue una progresión creciente acotada, partiendo desde un valor mínimo y acercándose gradualmente a un límite superior. Este crecimiento es suave y progresivo, permitiendo al jugador adaptarse en las primeras etapas y evitando aumentos bruscos de dificultad.

#### Densidad de enemigos

La frecuencia de aparición de enemigos también presenta una tendencia creciente acotada, pero su evolución no es completamente paralela a la velocidad. En particular, su incremento se encuentra desfasado o regulado, de modo que no coincida constantemente con los momentos de mayor velocidad.

#### Interacción y control dinámico de la dificultad

Si bien ambas variables aumentan con el tiempo, no deben entenderse como curvas independientes fijas, sino como parte de un sistema dinámico e interdependiente. En términos de diseño:

- La velocidad y la densidad de enemigos se ajustan entre sí para evitar picos simultáneos de dificultad.
- El sistema debe prevenir escenarios de saturación, donde alta velocidad y densidad ocurran al mismo tiempo de forma sostenida.
- Puede priorizarse el aumento de una variable mientras la otra se mantiene o crece más lentamente.

Este enfoque implica que la dificultad no responde únicamente a curvas asintóticas predefinidas, sino a una curva global dinámica, capaz de adaptarse al estado del juego en cada momento.

Finalmente, este sistema de progresión conjunta se ve posteriormente intervenido por la aparición del time-based boss, que actúa como elemento regulador del ritmo y de la acumulación de dificultad.

#### Sistema de recursos (camarones)

Los camarones recolectados se almacenan en un estado persistente del juego, permitiendo su uso tanto dentro como fuera de la partida, conectando progresión interna y externa.

## Sistema de puntaje

El puntaje se basa en el tiempo de supervivencia (centésimas de segundo), cumpliendo dos funciones:

- Medición del desempeño del jugador.
- Desbloqueo de contenido permanente.

## Diferenciadora

El núcleo identitario del juego es el sistema Ink-Pulse, una habilidad tipo dash que se recarga únicamente al exponerse al peligro (pasar cerca de amenazas sin colisionar). Esta lógica invierte el comportamiento tradicional del género: el progreso no se basa en evitar riesgos, sino en gestionarlos activamente.

Además, existen eventos que exigen su uso, integrando la mecánica dentro del ritmo del juego y evitando comportamientos pasivos. De este modo, el Ink-Pulse no es una ventaja opcional, sino un recurso central de supervivencia y decisión.

### ➤ Funcionalidad Técnica

#### Sistema de carga (graze zone)

El Ink-Pulse se modela mediante una barra de carga (slide bar) que se llena cuando el jugador permanece dentro de una zona de proximidad a amenazas (graze zone).

- Esta zona corresponde a una colisión superpuesta al personaje.
- No produce daño, pero detecta cercanía a enemigos/obstáculos.
- El tiempo acumulado dentro de esta zona incrementa la carga de la barra.

Esto permite cuantificar el riesgo asumido por el jugador de forma continua.

#### Activación del Ink-Pulse

Cuando la barra está completa, el jugador puede activar el Ink-Pulse (dash) mediante clic izquierdo.

El Ink-Pulse produce:

- Aumento temporal de velocidad (a través de la cámara de seguimiento).
- Duración fija ( $\approx 5$  segundos).
- Inmunidad a colisiones durante el efecto.

Funciona como una herramienta tanto de evasión como de superación de eventos críticos.

#### Impacto en la dificultad (sistema adaptativo)

El uso del Ink-Pulse no es neutro: tiene un efecto directo sobre el comportamiento del entorno.

#### Uso consistente y oportuno:

- Permite mantener el flujo del juego bajo control.
- Facilita la gestión de eventos críticos y densidad de amenazas.

#### Uso excesivamente conservador (no usarlo):

- El sistema incrementa la presión del entorno (mayor densidad o complejidad).
- Se generan situaciones donde el dash pasa de ser útil a obligatorio.

Esto introduce una penalización implícita a la pasividad.

## Time based Boss

Representa uno de los momentos de clímax dentro de la partida, interrumpiendo el flujo continuo del endless runner para introducir un desafío estructurado. Su propósito no es únicamente aumentar la dificultad, sino justificar mecánicamente el uso del Ink-Pulse, transformando la “pared” en una situación anticipable y coherente.

Este evento permite al jugador:

- Reconocer un punto crítico del ciclo de juego.
- Prepararse mentalmente para una decisión obligatoria.
- Validar el aprendizaje previo, especialmente en la gestión del riesgo.

El boss no es solo un obstáculo, sino un mecanismo de ritmo que organiza la experiencia en fases de tensión y resolución.

### ➤ Funcionalidad Técnica

#### Aparición controlada (basada en tiempo)

La aparición del time-based boss se rige por un intervalo base de tiempo, el cual se ajusta dinámicamente según el nivel de presión acumulada durante la partida.

En condiciones normales, el boss aparece cada cierto tiempo mínimo predefinido. Sin embargo, este intervalo se reduce progresivamente cuando el entorno se vuelve más exigente, particularmente en función de:

1. Aumento de la velocidad del juego, que incrementa la dificultad de reacción.
2. Mayor densidad de enemigos u obstáculos, que eleva la carga cognitiva y mecánica del jugador.

Este sistema debe cumplir las siguientes condiciones:

- **Intervalo base mínimo:** el boss no puede aparecer antes de un tiempo umbral, garantizando estabilidad en el ritmo inicial.
- **Ajuste dinámico:** a medida que la dificultad acumulada aumenta, el tiempo entre apariciones disminuye.
- **Dependencia del estado del juego:** la frecuencia de aparición responde directamente a variables del entorno (velocidad, densidad, intensidad).
- **Control del pacing:** la aparición del boss actúa como una interrupción deliberada de la escalada continua de dificultad, introduciendo un evento estructurado.
- **Equilibrio entre predictibilidad y adaptación:** no es completamente fijo ni aleatorio, sino coherente con la progresión del juego.

Este enfoque permite que el boss funcione como un mecanismo de regulación del ritmo, evitando saturación progresiva y aportando variedad controlada a la experiencia.

#### Cámara dinámica (zoom-out)

Durante el evento, la cámara realiza un zoom out controlado:

- Permite visualizar completamente al boss.
- Mejora la legibilidad de patrones y ataques.
- Refuerza la percepción de enfrentamiento significativo.

Tras el evento, la cámara retorna a su estado normal.

## Sistema de Ultimate (ataque “pared”)

El ataque principal del boss se estructura en tres fases claramente diferenciadas:

- Carga: El boss anticipa el ataque mediante señales visuales (Señales de advertencia). Permite al jugador prepararse y completar la carga de Ink-Pulse.
- Lanzamiento: Ventana temporal breve ( $\approx 2-3$  segundos) donde el jugador debe activar el dash. Es el momento de ejecución crítica.
- Resolución: Se despliega la “pared”.

Tras esto, el boss es retirado de escena, siendo arrastrado por la cámara, y el juego retoma su flujo normal.

## Impacto en la progresión (reset dinámico)

El time-based boss actúa como un punto de reinicio parcial del sistema de dificultad:

- Reduce la velocidad del entorno a un valor intermedio.
- Disminuye o reinicia la densidad de enemigos.
- Reinicia el “tiempo efectivo” de progresión.

Esto genera un ciclo estructural:

acumulación de dificultad  $\rightarrow$  clímax (boss)  $\rightarrow$  liberación  $\rightarrow$  reconstrucción

## Portales

Tras la finalización de un time-based boss, el jugador tiene la posibilidad de atravesar un portal de transición, que permite cambiar de zona o profundidad. Este sistema introduce variabilidad en el entorno, evitando la repetición prolongada de un mismo escenario y reforzando el dinamismo de la experiencia.

Narrativamente, el portal se justifica como una consecuencia del desbordamiento de amenazas, donde la presión acumulada obliga al protagonista a desplazarse hacia otra zona del ecosistema.

## Funcionalidad Técnica

### Aparición condicionada y probabilística

La aparición del portal está sujeta a dos reglas:

- Solo puede ocurrir inmediatamente después de un boss.
- Su generación responde a una función probabilística aleatoria “p”.

### Uso del portal

El acceso al portal requiere una acción deliberada:

- Posicionamiento dentro de la “zona” del portal.
- Activación del Ink-Pulse para atravesarlo.

### Efecto en el sistema de juego

Al atravesar el portal:

- Se produce un cambio de escenario o profundidad.
- Se reinician parcialmente variables del entorno (velocidad, densidad).
- Se inicia un nuevo ciclo de progresión.

## Gadgets y desbloqueables

Los gadgets son elementos obtenidos durante la partida a través del pez dealer, y tienen una duración limitada a la run en la que se adquieren. Su función es añadir valor inmediato a cada partida, introduciendo ventajas situacionales que obligan al jugador a tomar decisiones rápidas bajo presión.

A diferencia de mejoras permanentes, los gadgets refuerzan la idea de que cada run es única, ya que el jugador debe adaptarse a los recursos disponibles en ese momento.

### Funcionalidad técnica

#### Sistema de inventario

El jugador dispone de un inventario limitado de 2 slots, donde puede almacenar gadgets distintos. La gestión del inventario implica decisiones de reemplazo o conservación.

#### Sistema de obtención (pez dealer)

Los gadgets se obtienen mediante interacciones con el pez dealer:

- El jugador no conoce de antemano qué gadget recibirá.
- La asignación responde a un sistema aleatorio controlado.

Esto introduce incertidumbre y obliga a adaptarse a cada situación.

#### Sistema de desbloqueo progresivo

No todos los gadgets están disponibles desde el inicio:

- Algunos gadgets permanecen bloqueados.
- Se habilitan únicamente cuando el jugador alcanza ciertos umbrales de puntaje.

Esto vincula el rendimiento del jugador con la variedad de opciones disponibles.

## Tienda (Meta Game)

Permite al jugador invertir los camarones acumulados en bonificaciones permanentes o cosméticas. Entre estas se consideran mejoras como duplicación de recursos, aumento de la duración del Ink-Pulse y skins que modifican la apariencia del personaje y/o enemigos. Se accede a esta tienda tras finalizar una partida, sin restricciones de tiempo, y con disponibilidad constante de los objetos.



**Figura 9. DealerFish -> OctoDealer.**  
**Elaboración Propia.**

## Tienda (In-Run)

Durante la partida puede aparecer de forma no determinista un pez dealer, condicionado por el progreso del jugador (distancia o puntaje). Este permite comprar o intercambiar gadgets en tiempo real mediante un menú de duración limitada, obligando a tomar decisiones rápidas y evaluar el riesgo de desviarse de la trayectoria.

Su aparición no debe ser periódica ni predecible. En su lugar, debe cumplir las siguientes condiciones:

- **Cooldown mínimo:** no puede aparecer inmediatamente después de una aparición anterior.
- **Probabilidad creciente:** una vez superado ese umbral, la probabilidad de aparición aumenta gradualmente con el tiempo.
- **Frecuencia acotada:** existe un límite máximo para evitar apariciones excesivas.
- **Dependencia del progreso:** puede ajustarse según el desempeño del jugador.
- **Imprevisibilidad controlada:** evita tanto apariciones demasiado tempranas como esperas excesivas.

Este diseño mantiene el equilibrio entre pacing, economía y toma de decisiones bajo presión.

## Menú de opciones

Permite ajustar parámetros del juego, siendo clave para la accesibilidad y personalización. Incluye configuración de volumen (música y efectos), modo de pantalla (completa o ventana) y personalización de controles, especialmente para el uso de gadgets.

## Enemigos y obstáculos

Se consideran tres amenazas principales que estructuran el desafío del juego: pez globo, caña de pesca (anzuelo), mina submarina y SS Carnage. Cada una responde a un tipo de comportamiento distinto, generando variedad en la toma de decisiones del jugador.

### Tabla de Enemigos

| ELEMENTO       | TIPO DE COMPORTAMIENTO | FUNCIÓN EN JUGABILIDAD                  | CARACTERÍSTICA CLAVE                                       | PRE-DISEÑO                                                                            |
|----------------|------------------------|-----------------------------------------|------------------------------------------------------------|---------------------------------------------------------------------------------------|
| PEZ GLOBO      | Móvil (expansivo)      | Condiciona el espacio de desplazamiento | Reduce rutas de escape al aumentar su volumen              |  |
| ANZUELO        | Externo (dinámico)     | Introduce imprevisibilidad              | Irrumpe con trayectorias variables desde fuera del entorno |  |
| MINA SUBMARINA | Estático               | Castiga la imprecisión                  | Requiere control fino del movimiento en proximidad         |  |

**Tabla 4. Tabla de Enemigos. Elaboración Propia.****SS Carnage (Barco humano pesquero)****Rol narrativo**

Es la principal amenaza desde la perspectiva del protagonista, quien lo interpreta como el responsable del secuestro de su madre. Representa la intervención humana en el ecosistema marino y actúa como símbolo de explotación y peligro externo.

**Composición**

Está conformado por un grupo de pescadores que buscan capturar fauna marina. Va acompañado por un pato yeco (Yeico), que funciona como mascota y elemento distintivo del barco.

**Apariencia**

Barco pesquero grande, oxidado y deteriorado, cubierto de redes, arpones y herramientas de pesca. Los pescadores se ubican en la cubierta, visibles durante los ataques. Su tamaño es dominante, ocupando el borde superior de la pantalla.

**Participación en juego**

Funciona como un evento de alta presión que interrumpe el flujo normal de la partida, aumentando la intensidad del desafío.

**Ataque especial (Ultimate/Pared)**

Despliega una red de gran tamaño imposible de esquivar mediante movimiento convencional, obligando al uso del Ink-Pulse.

**Función de diseño**

Valida la mecánica central del juego al exigir el uso estratégico del Ink-Pulse, penalizando la falta de preparación y reforzando la identidad basada en riesgo y decisión.

**Figura 10. SS Carnage. Elaboración Propia.**



## Gadgets

Se consideran 2 gadgets a implementar, uno de ellos desbloqueable.

### Tabla de gadgets

| GADGET       | TIPO   | ACTIVACIÓN                 | EFEECTO PRINCIPAL                                 | CONDICIÓN DE APARICIÓN                       | PRE-DISEÑO                                                                          |
|--------------|--------|----------------------------|---------------------------------------------------|----------------------------------------------|-------------------------------------------------------------------------------------|
| SHELL SHIELD | Pasivo | Automática al recibir daño | Evita la derrota inmediata al absorber el impacto | Disponible tras superar los 5 minutos de run |  |
| INK BOTTLE   | Activo | Manual (teclas Q,W)        | Rellena instantáneamente la barra de Ink-Pulse    | Siempre disponible                           |  |

**Tabla 5. Tabla de Gadgets. Elaboración propia.**

## Estados del juego

El sistema se organiza en distintos estados que estructuran la experiencia y delimitan las interacciones del jugador:

- **Inicio / Menú principal:** acceso al juego, configuración básica y entrada a la partida.
- **Jugando (Gameplay):** estado principal, donde ocurre el desplazamiento continuo, la evasión de amenazas, la recolección de recursos y la gestión del *Ink-Pulse*.
- **Tienda (in-game):** espacio limitado en tiempo en medio de la run, que permite adquirir gadgets.
- **Game Over / Inconsciencia:** estado posterior a la colisión o fallo, acompañado de una breve resolución narrativa (rescate por parte de la madre).
- **Reintento:** transición rápida que permite reiniciar el ciclo sin fricción, reforzando la continuidad del juego.

## Condiciones

### Condición de progreso:

- El jugador avanza indefinidamente mientras logre evitar colisiones y gestionar correctamente sus recursos (Ink-Pulse y gadgets).

### Condición de derrota:

- Se produce al colisionar con un enemigo u obstáculo, o al no poder responder a eventos críticos que requieren el uso del Ink-Pulse. El resultado es el estado de inconsciencia del protagonista.

### Condición de uso crítico:

- Existen situaciones donde el uso del Ink-Pulse es obligatorio para continuar (por ejemplo, eventos tipo “pared”), lo que introduce una validación directa de la mecánica central.

## Ciclo de juego

El ciclo de juego se basa en una secuencia repetitiva de acciones: avanzar, esquivar, asumir riesgos para recargar *Ink-Pulse*, utilizarlo en momentos críticos, fallar o continuar, y reiniciar mediante el sistema de reintento.

## Diagrama de flujo

Squid Ink-Pulse - Flujo general del juego

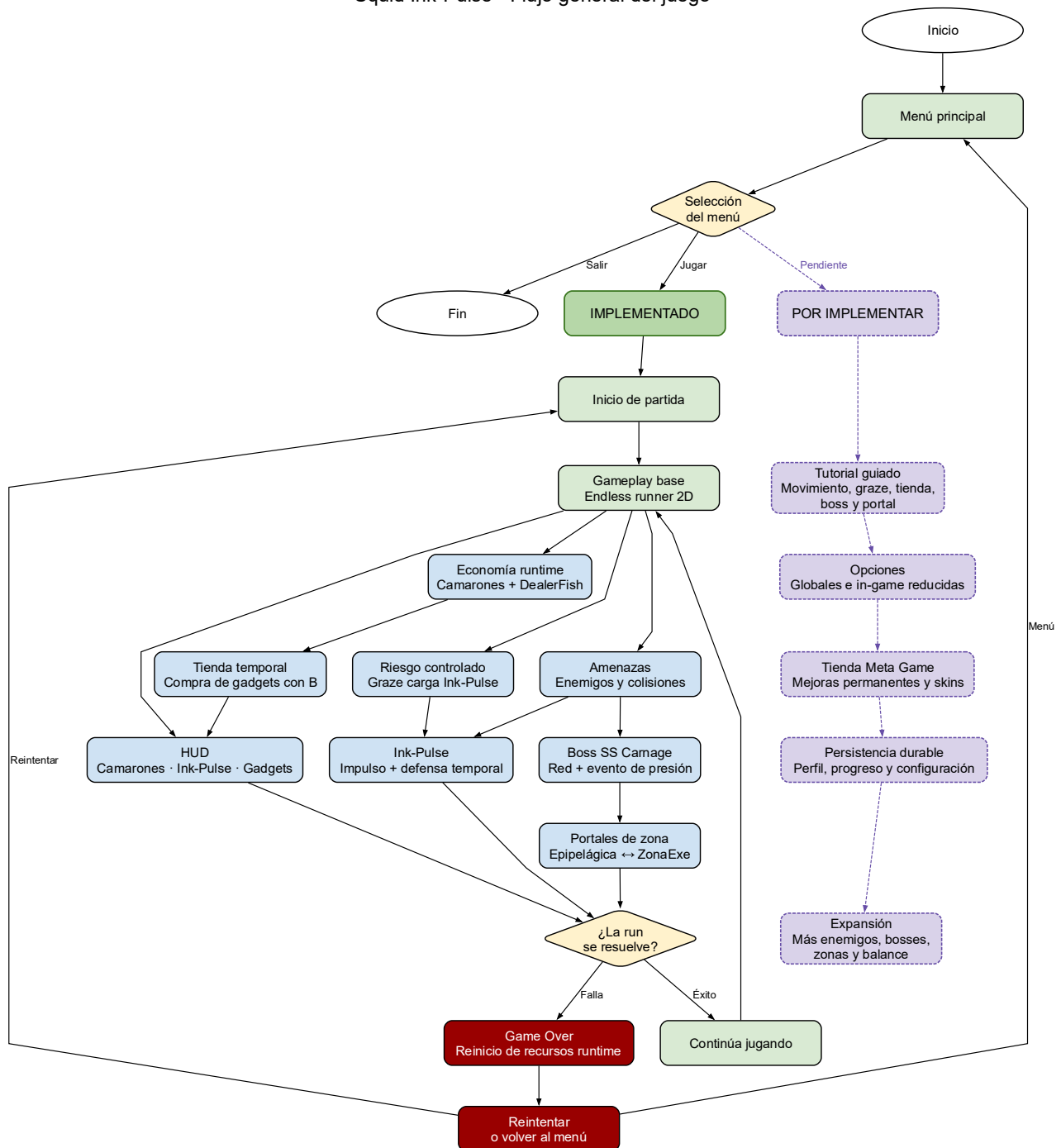


Figura 11. Diagrama de Flujo. Elaboración Propia.

## Sistema de progresión

La experiencia de juego está diseñada como un ciclo continuo de riesgo, recompensa y mejora, donde cada partida representa una oportunidad de superación. El jugador avanza enfrentando amenazas crecientes, recolectando recursos y tomando decisiones en tiempo real, lo que genera una sensación constante de tensión y dinamismo. Este ciclo se refuerza con el sistema de *Ink-Pulse*, que obliga a interactuar activamente con el peligro, evitando una jugabilidad pasiva.

### Progresión interna (In run)

Durante cada partida, la progresión se manifiesta a través de un aumento gradual de la dificultad:

- Incremento en la velocidad del juego.
- Mayor densidad y variedad de enemigos y obstáculos.
- Aparición de eventos críticos (como el S.S. Carnage).

Paralelamente, el jugador mejora su desempeño mediante:

- Recolección de camarones.
- Obtención y uso de gadgets.

Esta progresión genera una curva de aprendizaje inmediata, donde el jugador mejora dentro de la mismo run.

### Progresión externa (Metagaming)

Fuera de la partida, el jugador progresa mediante sistemas que incentivan la repetición:

- Uso de camarones como moneda acumulable.
- Acceso a mejoras y ventajas futuras.

Este nivel de progresión no solo mejora las capacidades del jugador, sino que también refuerza el compromiso a largo plazo, conectando múltiples partidas entre sí.

## Controles

El esquema de control es simple pero funcional, orientado a la rapidez de respuesta:

- **Movimiento:** mediante el mouse, Baby Squid siempre seguirá al puntero permitiendo desplazamiento vertical fluido.
- **Dash (Ink-Pulse):** activación con clic izquierdo, condicionado a la carga de la barra de adrenalina.
- **Gadgets:**
  - **Pasivos:** efectos permanentes o de activación con ciertos eventos ya implementados durante la partida.
  - **Activos:** uso manual asignado a teclas “Q,W”, permitiendo decisiones tácticas en tiempo real.
- **Otros:**
  - **Pausa:** Menú de pausa se activará con la tecla “P” y “Esc”.
  - **Manejo de tiendas:** Todo se hará mediante el mouse y el clic, y en in-game se permitirá el uso de la tecla “B”.

Este diseño mantiene una baja barrera de entrada, concentrando la complejidad en la gestión de recursos y la toma de decisiones bajo presión.

## Retención del jugador

La retención se basa en tres pilares principales:

- **Reintento inmediato:** la estructura del juego permite volver a jugar rápidamente tras perder, reduciendo la fricción.
- **Loop narrativo:** la intervención de la madre tras cada derrota justifica el reinicio, integrando narrativa y mecánica.
- **Progresión y dominio:** el jugador es incentivado a mejorar continuamente, ya sea superando su récord, optimizando el uso de recursos o dominando las mecánicas.

En conjunto, estos elementos generan una experiencia adictiva, coherente y orientada a la repetición, característica fundamental del género *endless runner*.

## Estado de cumplimiento

El estado actual del proyecto puede describirse como un MVP funcional en cumplimiento parcial. El juego ya cuenta con un núcleo jugable operativo: movimiento continuo, control del jugador, sistema Ink-Pulse, detección de proximidad o *graze*, recolección de camarones, gadgets, tienda temporal, enemigos, portales y evento de boss. Esto permite validar la propuesta central del juego: avanzar, esquivar, asumir riesgos, cargar Ink-Pulse y usarlo en momentos críticos.

### Resumen de cumplimiento por componente

| COMPONENTE DEL MVP      | ESTADO   | EVIDENCIA DE AVANCE                                                   | OBSERVACIÓN                                                       |
|-------------------------|----------|-----------------------------------------------------------------------|-------------------------------------------------------------------|
| MOVIMIENTO DEL JUGADOR  | Cumplido | Movimiento continuo, control vertical y límites de desplazamiento.    | Requiere ajustes finos de sensibilidad y balance.                 |
| INK-PULSE               | Cumplido | Sistema de carga, activación, estados y reinicio en Game Over.        | Es el mayor avance identitario del proyecto.                      |
| GRAZE ZONE / PROXIMIDAD | Cumplido | La cercanía a amenazas carga Ink-Pulse sin requerir colisión.         | Debe probarse para evitar que sea demasiado fácil o frustrante.   |
| CAMARONES               | Parcial  | Recolección, valor de moneda y visualización en HUD.                  | Falta persistencia permanente fuera de runtime.                   |
| GADGETS E INVENTARIO    | Cumplido | Inventario por slots, gadgets activos y pasivos, uso con teclas.      | Falta balancear frecuencia, costo e impacto real.                 |
| TIENDA IN-GAME          | Cumplido | Pez dealer, oferta temporal, precio, contador y compra con camarones. | Falta ajustar aparición y relación con dificultad.                |
| ENEMIGOS COMUNES        | Parcial  | Spawn, tags, pez globo, mina y caña de pescar.                        | Algunos comportamientos aún requieren mayor desarrollo o balance. |
| SS CARNAGE              | Cumplido | Boss con fases, red, advertencia, resolución y fallo.                 | Requiere pulido visual, sonoro y pruebas de dificultad.           |
| PORTALES                | Parcial  | Cambio entre zona epipelágica y zona abisopelágica.                   | Falta mejorar feedback y diferenciación jugable.                  |
| HUD Y MENÚS             | Parcial  | HUD, pausa, game over y tienda temporal implementados.                | Faltan opciones globales, configuración y pulido de UI.           |

**Tabla 6. Elaboración Propia.**

## Visualización de implementaciones



Figura 12. Menú Principal.



Figura 13. Menú de Pausa.

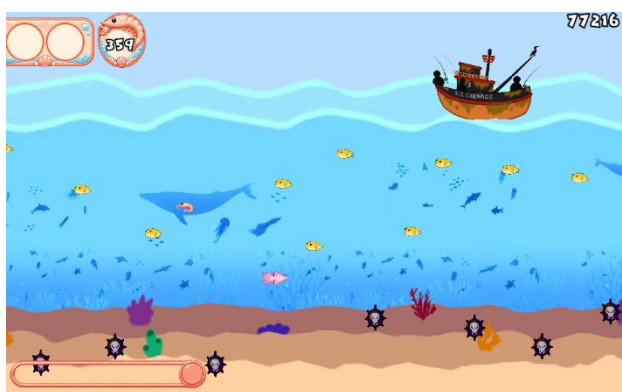


Figura 14. Aparición del Carnage

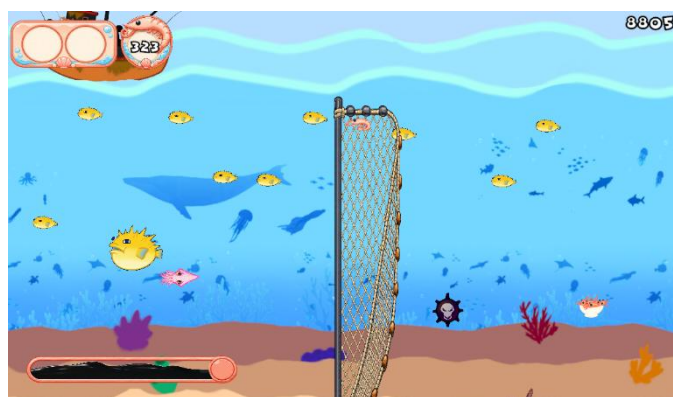


Figura 15. Evento de "pared" del SS Carnage.

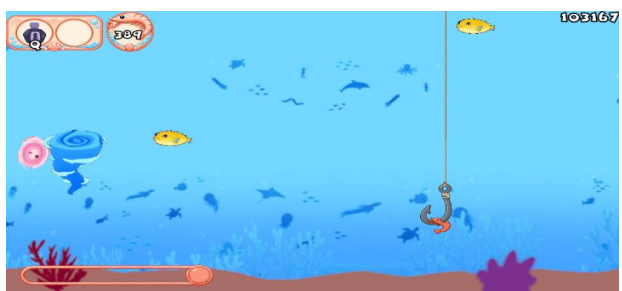


Figura 16. Portales en forma de remolinos.

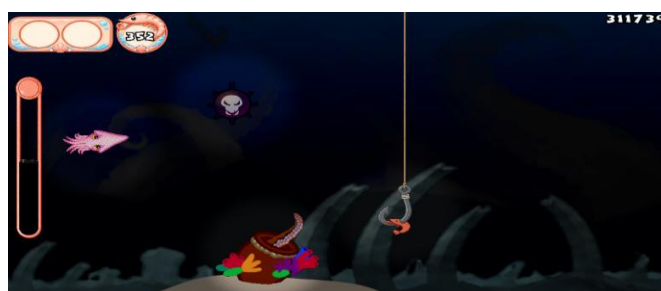


Figura 17. Zona abisopelágica y OctoDealer "in



Figura 18. Menú de tienda in run.

## Pendientes principales

| ÁREA PENDIENTE            | QUÉ FALTA CERRAR                                                               | PRIORIDAD |
|---------------------------|--------------------------------------------------------------------------------|-----------|
| <b>PERSISTENCIA</b>       | Guardar camarones, progreso y estado del jugador fuera de runtime.             | Alta      |
| <b>TIENDA GLOBAL</b>      | Implementar tienda fuera de la partida para mejoras permanentes.               | Alta      |
| <b>TUTORIAL</b>           | Enseñar movimiento, graze, Ink-Pulse, camarones y gadgets.                     | Alta      |
| <b>BALANCE</b>            | Ajustar velocidad, spawn, dificultad del boss, economía y aparición de tienda. | Alta      |
| <b>DESBLOQUEABLES</b>     | Agregar las condiciones o hitos de desbloqueo permanente.                      | Alta      |
| <b>QA FORMAL</b>          | Registrar pruebas, errores, correcciones y validación de mecánicas.            | Media     |
| <b>OPCIONES</b>           | Incorporar volumen, pantalla y configuración básica.                           | Media     |
| <b>PULIDO AUDIOVISUAL</b> | Mejorar feedback de portales, boss, tienda, zona oscura y HUD.                 | Media     |

**Tabla 7. Elaboración Propia.**

El proyecto presenta un avance sustantivo. La propuesta pasó desde una definición conceptual y un prototipo incompleto hacia una versión jugable con sistemas centrales conectados. El principal logro corresponde a la implementación del Ink-Pulse y del graze, ya que ambos sistemas materializan la identidad del juego basada en riesgo y recompensa.

El cumplimiento general puede considerarse alto, porque el núcleo del MVP ya permite representar la experiencia principal del juego. No obstante, aún quedan pendientes técnicos y de diseño necesarios para cerrar la entrega: persistencia, tienda global, tutorial, balance, opciones y QA.

## Desarrollo

El desarrollo de *Squid Ink-Pulse* se estructura como un proyecto de videojuego 2D desarrollado en Unity, orientado al género *endless runner* con énfasis en acción, riesgo y progresión. El proyecto no se limita a implementar una escena jugable aislada, sino que organiza sus sistemas principales en torno a un ciclo de juego persistente: el jugador controla a Baby Squid, avanza de forma continua, esquiva amenazas, recolecta camarones, carga el recurso Ink-Pulse mediante exposición al riesgo y utiliza dicho impulso para sobrevivir o superar eventos críticos.

Desde el punto de vista del desarrollo, el repositorio evidencia una evolución desde una idea base de juego rápido y reactivo hacia una implementación más estructurada. El proyecto incorpora sistemas de sesión, progresión de partida, movimiento del jugador, enemigos, tienda temporal, gadgets, portales entre zonas, HUD, menús, persistencia local y documentación técnica. Esto permite considerar el desarrollo como un MVP funcional en expansión, donde varias mecánicas centrales ya se encuentran formalizadas y otras quedan preparadas para iteraciones futuras.

La estructura general del proyecto diferencia claramente entre implementación técnica, contenido de juego y documentación. Esta separación favorece el mantenimiento, ya que evita mezclar código fuente, assets visuales, prefabs, escenas y documentos técnicos en un mismo nivel de responsabilidad. En consecuencia, el desarrollo puede ser comprendido como un trabajo modular: cada sistema se implementa, prueba y documenta dentro de su propio dominio.

| ÁREA                         | FUNCIÓN DENTRO DEL DESARROLLO                                             |
|------------------------------|---------------------------------------------------------------------------|
| ASSETS/IMPLEMENTATION/       | Contiene el código C#, configuraciones técnicas y herramientas de editor. |
| ASSETS/CONTENT/              | Agrupar prefabs, arte, audio y animaciones utilizados en runtime.         |
| ASSETS/SCENES/               | Contiene las escenas jugables y de menú.                                  |
| ASSETS/STREAMINGASSETS/DB/   | Almacena semillas JSON utilizadas para perfil, catálogo y récords.        |
| DOCS/                        | Contiene la documentación técnica viva del proyecto.                      |
| PACKAGES/ Y PROJECTSETTINGS/ | Mantienen la configuración base del proyecto Unity.                       |

**Tabla 8. Elaboración Propia.**

En términos de avance, el proyecto presenta un desarrollo orientado a sistemas reutilizables. En vez de resolver cada mecánica con scripts aislados, se observa una tendencia a centralizar responsabilidades: la sesión global se controla desde un sistema de sesión, la progresión de dificultad desde un director de run, los cambios de escena desde un controlador de flujo, y la generación de objetos desde un spawner. Esta organización reduce la duplicación de lógica y permite que las escenas compartan reglas comunes.



## Programación

La programación del proyecto está realizada principalmente en C# sobre Unity. El código se organiza bajo Assets/Implementation/Code/, donde cada carpeta representa un dominio funcional del juego. Esta decisión permite que el proyecto sea más legible y mantenible, ya que cada módulo concentra una responsabilidad específica.

| CARPETA     | RESPONSABILIDAD PRINCIPAL                                                                               |
|-------------|---------------------------------------------------------------------------------------------------------|
| CORE/       | Sesión global, progresión de run, control de escenas, cámara, boundaries e infraestructura transversal. |
| PLAYER/     | Movimiento, Ink-Pulse, colisiones, interacción, inventario, visuales y perfil persistente.              |
| SPAWNING/   | Generación de enemigos, camarones, portales y tienda temporal.                                          |
| ENEMIES/    | Comportamientos específicos de enemigos.                                                                |
| BOSSSES/    | Lógica de eventos de jefe y comportamientos asociados al boss.                                          |
| UI/         | HUD, pausa, game over, tienda, displays y animación de interfaz.                                        |
| WORLD/      | Elementos de mundo como portales, tienda in-game e iluminación por zona.                                |
| AUDIO/      | Música dinámica, efectos sonoros y transiciones de audio.                                               |
| BACKGROUND/ | Parallax y fondos.                                                                                      |
| MAINMENU/   | Navegación del menú principal.                                                                          |
| TUTORIAL/   | Flujo y pasos del tutorial.                                                                             |

**Tabla 9. Elaboración Propia.**

El sistema del jugador está dividido en componentes especializados para evitar que un solo script concentre demasiadas responsabilidades. PlayerMovement gestiona el desplazamiento y la respuesta al mouse; InkPulseController controla la carga, activación y duración del Ink-Pulse; GrazeDetector permite cargar el recurso al pasar cerca de amenazas; y PlayerCollision resuelve las interacciones con enemigos, camarones, portales u otros objetos relevantes.

La mecánica de Ink-Pulse se programa mediante una máquina de estados con fases como Idle, Charging, Ready y Active. Esto permite controlar con mayor claridad cuándo el recurso puede cargarse, activarse o bloquearse por situaciones como tienda abierta, transición de portal, muerte o Game Over.

Por otro lado, la progresión de la partida se separa del movimiento del jugador mediante RunProgressionDirector, que regula intensidad, velocidad, ritmo de aparición de obstáculos y eventos de boss. La generación de entidades queda a cargo de LevelSpawner, que instancia enemigos, camarones, DealerFish y portales según perfiles de zona, delegando parte de su lógica en servicios internos para mantener el sistema ordenado y extensible.

## Arquitectura

La arquitectura de software de *Squid Ink-Pulse* se organiza como una arquitectura modular por dominios, implementada sobre el modelo de componentes propio de Unity. Esto significa que el proyecto no se estructura como un único bloque de scripts dependientes entre sí, sino como un conjunto de dominios funcionales separados, cada uno con una responsabilidad clara dentro del videojuego.

En este contexto, el término dominio se entiende como una agrupación técnica y funcional dentro de `Assets/Implementation/Code/`. Cada dominio concentra un área específica del sistema: Core contiene la lógica transversal de sesión, escenas, cámara y progresión; Player agrupa movimiento, Ink-Pulse, colisiones, inventario, visuales y perfil persistente; Spawning administra la aparición de enemigos, camarones, portales y tienda temporal; UI contiene HUD, menús y displays; World reúne entidades del entorno como portales, DealerFish e iluminación; y otros dominios como Audio, Background, Enemies, Bosses, Tutorial y MainMenu aíslan responsabilidades complementarias.

La decisión de organizar el código por dominios permite que el proyecto sea más mantenible y escalable. Si se modifica el sistema de aparición de enemigos, el cambio se concentra principalmente en Spawning; si se ajusta el comportamiento del jugador, el trabajo se realiza en Player; si se modifica el HUD o los menús, la responsabilidad corresponde a UI. Esta organización reduce el acoplamiento entre sistemas y evita que la lógica de juego quede dispersa en scripts genéricos o en prefabs sin responsabilidad definida.

La jerarquía arquitectónica principal puede representarse de la siguiente forma:

| NIVEL                        | ROL DENTRO DE LA ARQUITECTURA                                                                       | EJEMPLOS                                             |
|------------------------------|-----------------------------------------------------------------------------------------------------|------------------------------------------------------|
| DOMINIO                      | Agrupar una responsabilidad funcional del juego.                                                    | Core, Player, Spawning, UI, World, Audio.            |
| ORQUESTADORES / CONTROLLERS  | Gobiernan un sistema, coordinan referencias, ejecutan transiciones y exponen parámetros de balance. | RunProgressionDirector, InkPulseController.          |
| ESTADOS FORMALES             | Representan fases discretas del sistema sin depender directamente de Unity ni de prefabs.           | GameSessionState, PlayerRuntimeState, InkPulseState. |
| ESPECIALIZACIONES            | Implementan comportamientos concretos y limitados dentro de un dominio.                             | PufferfishEnemy, ScenePortal.                        |
| DATOS, CATÁLOGOS Y SERVICIOS | Almacenan configuración, persistencia, reglas de selección, cálculo o adaptación.                   | GadgetCatalog, EnemySpawnSelector.                   |

**Tabla 10. Elaboración Propia.**

La dirección correcta de dependencia parte desde el dominio hacia sus componentes internos. El dominio actúa como contenedor conceptual; dentro de él, los orquestadores concentran la autoridad del sistema. Los estados formales no ejecutan por sí mismos la lógica de Unity, sino que describen la fase en la que se encuentra un sistema. Luego, las especializaciones ejecutan comportamientos concretos, pero no deberían tomar decisiones globales. Finalmente, los datos, catálogos, perfiles, repositorios y servicios entregan información o cálculos auxiliares sin apropiarse del flujo principal del juego.

Por ejemplo, LevelSpawner funciona como orquestador del dominio Spawning: decide cuándo y dónde instanciar enemigos, camarones, portales o eventos de tienda. Sin embargo, no debería contener el

comportamiento particular de cada enemigo. Esa lógica corresponde a especializaciones como `PufferfishEnemy` o `FishingRodEnemy`. De la misma forma, un `ScenePortal` puede detectar una interacción con el jugador, pero no debe decidir por sí mismo toda la política de flujo de escenas; esa responsabilidad pertenece a los controladores de escena y sesión.

El proyecto evidencia patrones de diseño como **State Machine**, usado para representar estados formales del juego; **Facade**, aplicado en objetos raíz que agrupan referencias y simplifican la composición de escena. Además, incorpora servicios internos y una lógica *data-driven*, mediante selectores, resolvers, calculadores, perfiles y catálogos, lo que permite reducir la carga de los orquestadores, ajustar parámetros de balance sin modificar la lógica central y mantener una separación clara entre autoridad, estado, comportamiento y configuración.

## Persistencia

Esto está en fase de implementación, sin embargo, ya existe programación orientada a persistencia local mediante JSON. El sistema de perfil persistente permite guardar camarones, récords, skins, mejoras permanentes, desbloques de gadgets. Esto introduce una diferencia clara entre estado runtime y estado permanente: los gadgets comprados durante una run se reinician al perder, mientras que la economía, récords y desbloques permanecen asociados al perfil del jugador.

## Documentación

El repositorio presenta una documentación técnica amplia y organizada bajo la carpeta `Docs/`. Esta documentación funciona como contrato vivo del proyecto, ya que describe arquitectura, estructura de carpetas, sistemas de gameplay, enemigos, UI, QA, persistencia, portales, cámara, mundo y roadmap. Su existencia permite que el informe no dependa únicamente de una descripción externa, sino de documentos internos que explican cómo debe crecer el código.

La documentación cumple tres funciones principales. Primero, registra decisiones de arquitectura, como la separación entre dominios, estados y especializaciones. Segundo, define contratos técnicos de escena y prefab, indicando qué nodos deben existir, qué scripts pertenecen a cada sistema y qué referencias deben resolverse por Inspector o por contrato. Tercero, permite orientar el testing, ya que identifica parámetros ajustables, reglas no balanceables y condiciones que deben validarse antes de considerar estable una escena.

| DOCUMENTO TÉCNICO              | APORTE AL PROYECTO                                                                                  |
|--------------------------------|-----------------------------------------------------------------------------------------------------|
| <b>SOFTWAREARCHITECTURE.MD</b> | Define la arquitectura, capas, dependencias, reglas de nomenclatura y criterios de refactorización. |
| <b>PROJECTSTRUCTURE.MD</b>     | Explica la organización de carpetas y la responsabilidad de cada dominio.                           |
| <b>STATEMACHINES.MD</b>        | Registra las máquinas de estado formales del juego.                                                 |
| <b>QATESTER.MD</b>             | Define parámetros de prueba, validaciones y metodología de testing.                                 |

**Tabla 11. Elaboración Propia.**

Desde una perspectiva académica, esta documentación evidencia buenas prácticas de ingeniería de software: separación de responsabilidades, trazabilidad de decisiones, control de cambios y criterios para futuras refactorizaciones. Además, al existir documentación específica para QA, el proyecto no solo describe cómo funciona, sino también cómo debe probarse.

La documentación también cumple una función metodológica. Permite que nuevos integrantes comprendan el proyecto sin depender exclusivamente de comunicación oral o revisión manual de escenas Unity. Esto es especialmente importante en un proyecto de equipo, donde programación, diseño, arte, sonido y testing deben coordinarse sobre una misma base técnica.

## Testing

El testing del proyecto debe entenderse como un proceso de validación funcional, técnica y de balance. No basta con comprobar que el juego “abre” o que el jugador se mueve; es necesario verificar que cada sistema respete su contrato y que los cambios de balance no rompan la arquitectura general.

El primer nivel de prueba corresponde a la validación de contratos de escena. Antes de balancear dificultad, enemigos o recompensas, debe comprobarse que las escenas contengan los nodos obligatorios: boundaries del jugador y cámara, prefabs correctos, tags, layers, HUD, managers, spawner, portales y referencias necesarias. Si una escena está mal cableada, el error debe corregirse como problema técnico antes de interpretarlo como problema de gameplay.

El segundo nivel corresponde al testing funcional. Aquí se revisa que cada mecánica responda correctamente:

| SISTEMA PROBADO | CRITERIO DE VALIDACIÓN                                                                        |
|-----------------|-----------------------------------------------------------------------------------------------|
| MOVIMIENTO      | El jugador avanza correctamente, responde al mouse y respeta límites verticales.              |
| INK-PULSE       | Carga con graze, pasa a Ready, se activa con input válido y se reinicia en Game Over.         |
| COLISIONES      | Las amenazas producen derrota salvo que exista protección válida.                             |
| CAMARONES       | Se recolectan, actualizan HUD y persisten en la economía.                                     |
| GADGETS         | Se compran, ocupan slots, se activan con Q/W si corresponde y se reinician al perder.         |
| TIENDA TEMPORAL | Aparece mediante DealerFish, muestra oferta, calcula precio y cierra.                         |
| SPAWNING        | Genera enemigos, camarones, tienda y portales según intensidad y perfil de zona.              |
| PORTALES        | Cambian de zona, conservan gadgets e Ink-Pulse y no provocan Game Over.                       |
| UI              | Muestra barra de Ink-Pulse, score, camarones, gadgets, pausa y derrota sin gobernar gameplay. |

**Tabla 12. Elaboración Propia.**

El tercer nivel corresponde al testing de balance. En este caso se deben modificar parámetros uno por uno, registrar el valor anterior, el valor nuevo y el efecto observado. Esto es fundamental porque si se alteran varios valores al mismo tiempo no es posible atribuir el resultado a una causa específica. Los parámetros más relevantes para balance son la velocidad de progresión, intervalos de spawn, frecuencia de enemigos, probabilidad de camarones, aparición de tienda, duración de ofertas, precios, duración del Ink-Pulse y ventanas de portal.

El cuarto nivel corresponde al testing de persistencia. Debe verificarse que los datos permanentes se mantengan entre sesiones y que los datos runtime se limpien cuando corresponde. Por ejemplo, los camarones y récords deben conservarse, pero los gadgets comprados durante una run no deben permanecer después de Game Over. Esta distinción es clave para evitar errores de economía o ventajas no previstas.

Finalmente, el testing debe incluir pruebas de regresión cada vez que se modifique una escena, prefab o sistema central. Una modificación en el jugador puede afectar movimiento, colisión, graze, Ink-Pulse, portales, HUD y Game Over. Del mismo modo, un cambio en LevelSpawner puede afectar enemigos, tienda, camarones, portales y balance de dificultad. Por ello, el proyecto requiere pruebas cruzadas entre sistemas, no solo pruebas aisladas por script.

En síntesis, el testing de *Squid Ink-Pulse* debe seguir una lógica ordenada: primero validar contratos técnicos, luego comprobar funcionamiento, después ajustar balance y finalmente verificar persistencia y regresión. Esta metodología reduce errores acumulativos y permite que el proyecto avance de forma controlada hacia una versión más estable.

## Análisis de costos

Para analizar la viabilidad económica del proyecto, se elaboraron dos planillas de costos: una correspondiente al MVP académico y otra al proyecto ideal comercial. Esta separación permite diferenciar el alcance real del trabajo desarrollado durante el semestre de una proyección más amplia orientada a una eventual publicación del videojuego.

El MVP académico considera el período real de desarrollo, desde marzo de 2026 hasta la primera semana de julio de 2026. Este escenario valoriza el trabajo necesario para construir una versión mínima funcional del juego. En cambio, el proyecto ideal comercial considera un ciclo anual de desarrollo, con mayor cantidad de contenido, mayor nivel de pulido, publicación, marketing y pruebas más extensas.

### Escenarios considerados

| ESCENARIO                       | PLAZO CONSIDERADO                         | PROPÓSITO                                                                |
|---------------------------------|-------------------------------------------|--------------------------------------------------------------------------|
| <b>MVP ACADÉMICO</b>            | Marzo 2026 – primera semana de julio 2026 | Escenario ideal para validar una versión mínima funcional del videojuego |
| <b>PROYECTO IDEAL COMERCIAL</b> | 12 meses estimados                        | Proyectar una versión completa, pulida y publicable                      |

*Tabla 13. Elaboración Propia.*

### Resumen comparativo de costos

| ESCENARIO                       | TOTAL ESTIMADO    | DESCRIPCIÓN                                                                                |
|---------------------------------|-------------------|--------------------------------------------------------------------------------------------|
| <b>MVP ACADÉMICO</b>            | \$6.389.625 CLP   | Valor referencial del trabajo y recursos necesarios para una versión mínima funcional      |
| <b>PROYECTO IDEAL COMERCIAL</b> | \$211.810.220 CLP | Estimación de una versión comercial completa con mayor producción, publicación y marketing |
| <b>DIFERENCIA</b>               | \$205.420.595 CLP | Aumento asociado al mayor alcance, duración y especialización del proyecto ideal           |

*Tabla 14. Elaboración Propia.*

## Elementos considerados en ambos escenarios

En ambas planillas se consideraron costos asociados al desarrollo del videojuego, pero ajustados al alcance de cada escenario. La diferencia principal está en la profundidad con que se aborda cada área.

| ÁREA            | CONSIDERACIÓN EN MVP                                  | CONSIDERACIÓN EN PROYECTO IDEAL                     |
|-----------------|-------------------------------------------------------|-----------------------------------------------------|
| PROGRAMACIÓN    | Mecánicas centrales funcionales                       | Sistemas completos, optimizados y escalables        |
| ARTE 2D         | Assets básicos para representar el juego              | Arte final, animaciones y mayor coherencia visual   |
| DISEÑO DE JUEGO | Mecánica principal, flujo básico y dificultad inicial | Balance completo, progresión y economía interna     |
| AUDIO           | Efectos básicos o recursos simples                    | Música y efectos originales                         |
| TESTING         | Pruebas funcionales mínimas                           | QA sistemático, pruebas de balance y rendimiento    |
| DOCUMENTACIÓN   | Informe, planificación y respaldo técnico             | Documentación de producción y preparación comercial |

**Tabla 15. Elaboración Propia.**

## Criterios utilizados para la estimación

Las estimaciones consideran principalmente los recursos humanos involucrados en el desarrollo, además de las tareas asociadas a programación, diseño, arte, pruebas y documentación. En ambos escenarios se valoriza el tiempo de trabajo requerido para desarrollar el videojuego, ajustando el presupuesto según el alcance esperado.

Para el cálculo del MVP se consideraron únicamente los elementos necesarios para construir una versión funcional capaz de demostrar la propuesta principal del juego. Esto incluye el desarrollo de las mecánicas centrales, la implementación de una interfaz básica, la creación de recursos visuales mínimos, pruebas funcionales y la documentación requerida para el proyecto académico.

Por otro lado, el proyecto ideal incorpora una visión más amplia del desarrollo, considerando una mayor duración del proyecto, un nivel superior de producción artística y sonora, procesos de prueba más completos, actividades de publicación y acciones de difusión orientadas a una eventual comercialización.

## Desglose de costos: MVP

| C<br>A<br>J<br>A                |                                         |              | Marzo         | Abril       | Mayo        | Junio        | Julio         |
|---------------------------------|-----------------------------------------|--------------|---------------|-------------|-------------|--------------|---------------|
|                                 | CAJA INICIAL                            |              |               |             |             |              |               |
|                                 | TOTAL INGRESO                           |              | \$ 6.389.625  | \$ -        | \$ -        | \$ -         | \$ -          |
|                                 | TOTAL EGRESO                            |              | \$ -3.125.000 | \$ -775.000 | \$ -675.000 | \$ -775.000  | \$ -1.039.625 |
|                                 | ESTADO DE CAJA                          |              | \$ 3.264.625  | \$ -775.000 | \$ -675.000 | \$ -775.000  | \$ -1.039.625 |
|                                 |                                         |              |               |             |             |              |               |
| I<br>N<br>G<br>R<br>E<br>S<br>O | Ingresos                                | Acumulado    | Marzo         | Abril       | Mayo        | Junio        | Julio         |
|                                 | FONDO SOLICITADO                        | \$ 6.389.625 | \$ 6.389.625  |             |             |              |               |
|                                 |                                         | \$ -         |               |             |             |              |               |
|                                 |                                         | \$ -         |               |             |             |              |               |
|                                 | TOTAL                                   | \$ 6.389.625 | \$ 6.389.625  | \$ -        | \$ -        | \$ -         | \$ -          |
|                                 |                                         |              |               |             |             |              |               |
| S<br>U<br>E<br>L<br>D<br>O<br>S | Sueldos                                 | Acumulado    | Marzo         | Abril       | Mayo        | Junio        | Julio         |
|                                 | Artista 2D externo                      | \$ 450.000   |               | \$450.000   |             |              |               |
|                                 | Apoyo UI / interfaz                     | \$ 200.000   |               |             | \$200.000   |              |               |
|                                 | Animación 2D reforzada                  | \$ 600.000   |               |             | \$300.000   | \$300.000    |               |
|                                 | Música y efectos sonoros básicos        | \$ 150.000   |               |             |             | \$150.000    |               |
|                                 |                                         | \$ -         |               |             |             |              |               |
|                                 | TOTAL                                   | \$ 1.400.000 | \$ -          | \$ 450.000  | \$ 500.000  | \$ 450.000   | \$ -          |
|                                 |                                         |              |               |             |             |              |               |
| A<br>C<br>T<br>I<br>V<br>O<br>S | Gastos de activos                       | Acumulado    | Marzo         | Abril       | Mayo        | Junio        | Julio         |
|                                 | Computadores aptos para Unity           | \$ 2.750.000 | \$2.750.000   |             |             |              |               |
|                                 | Recursos gráficos, assets y paquetes 2D | \$ 250.000   |               | \$250.000   |             |              |               |
|                                 | Software de arte, audio o edición       | \$ 200.000   | \$200.000     |             |             |              |               |
|                                 | Almacenamiento, respaldo y gestión      | \$ 100.000   | \$100.000     |             |             |              |               |
|                                 | Recursos técnicos menores               | \$ 100.000   |               |             | \$100.000   |              |               |
|                                 |                                         | \$ -         |               |             |             |              |               |
|                                 |                                         | \$ -         |               |             |             |              |               |
|                                 |                                         | \$ -         |               |             |             |              |               |
| TOTAL                           | \$ 3.400.000                            | \$ 3.050.000 | \$ 250.000    | \$ 100.000  | \$ -        | \$ -         |               |
|                                 |                                         |              |               |             |             |              |               |
| A<br>D<br>M<br>I<br>N           | Gastos Administrativos                  | Acumulado    | Marzo         | Abril       | Mayo        | Junio        | Julio         |
|                                 | Gestión y coordinación                  | \$ 300.000   | \$75.000      | \$75.000    | \$75.000    | \$75.000     |               |
|                                 | Testing funcional                       | \$ 250.000   |               |             |             | \$150.000    | \$100.000     |
|                                 | Documentación e informe                 | \$ 250.000   |               |             |             | \$100.000    | \$150.000     |
|                                 | Integración y cierre de build           | \$ 208.750   |               |             |             |              | \$208.750     |
|                                 | Contingencia                            | \$ 580.875   |               |             |             |              | \$580.875     |
|                                 |                                         | \$ -         |               |             |             |              |               |
| TOTAL                           | \$ 1.589.625                            | \$ 75.000    | \$ 75.000     | \$ 75.000   | \$ 325.000  | \$ 1.039.625 |               |

**Tabla 16. Desglose de costos. Elaboración Propia.**

Estos costos están fundamentados en las referencias adjuntas al final del informe.



## Elementos no considerados en el MVP

El MVP académico no incorpora ciertos costos debido a que su objetivo es validar la jugabilidad principal del proyecto dentro del contexto del semestre. Por esta razón, se excluyeron elementos que no son indispensables para demostrar el funcionamiento del videojuego.

Esta decisión permite mantener una estimación coherente con los objetivos académicos del proyecto y evita incorporar costos que no aportan directamente a la validación de la propuesta jugable.

| ELEMENTO EXCLUIDO DEL MVP             | MOTIVO DE EXCLUSIÓN                                                           |
|---------------------------------------|-------------------------------------------------------------------------------|
| PUBLICACIÓN COMERCIAL                 | La entrega académica no requiere lanzamiento en plataformas digitales         |
| MARKETING Y PUBLICIDAD                | No forman parte de los objetivos de validación del proyecto                   |
| SERVIDORES O INFRAESTRUCTURA ONLINE   | El alcance actual no contempla funcionalidades multijugador                   |
| ACTORES DE VOZ O DOBLAJE              | No son necesarios para la experiencia principal                               |
| LICENCIAS PROFESIONALES DE ALTO COSTO | Se prioriza el uso de herramientas gratuitas o disponibles institucionalmente |
| QA PROFESIONAL EXTERNO                | Las pruebas son realizadas por el propio equipo                               |
| CONTRATACIÓN DE PERSONAL EXTERNO      | El desarrollo es realizado por los integrantes del proyecto                   |
| CAMPAÑAS DE LANZAMIENTO               | Corresponden a una etapa posterior de comercialización                        |

**Tabla 17. Elaboración Propia.**

El proyecto ideal comercial corresponde a una proyección ampliada de Squid Ink-Pulse, considerando una versión completa, pulida y preparada para una eventual publicación. A diferencia del MVP académico, este escenario incorpora mayor duración de desarrollo, equipo profesional, apoyo externo especializado, equipamiento, herramientas, marketing y contingencia.

| BLOQUE DE COSTO                         | MONTO ESTIMADO    | EXPLICACIÓN BREVE                                                                                                                                                                                                                                    |
|-----------------------------------------|-------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| EQUIPO PROFESIONAL Y DESARROLLO TÉCNICO | \$105.180.000 CLP | Corresponde al equipo base necesario para desarrollar el juego de forma profesional: productor, diseñadores, programadores y QA. Este bloque cubre la construcción de mecánicas, tienda, gadgets, enemigos, interfaz, pruebas y estabilidad general. |
| PRODUCCIÓN VISUAL, ANIMACIÓN Y AUDIO    | \$35.420.000 CLP  | Incluye la creación de identidad visual y sonora del juego: arte 2D, animaciones, UI, música y efectos. Permite que el juego no solo funcione, sino que tenga una presentación más pulida y coherente.                                               |
| EQUIPAMIENTO, HERRAMIENTAS Y LICENCIAS  | \$26.110.000 CLP  | Considera computadores, periféricos, software, licencias y recursos técnicos necesarios para trabajar en Unity, producir assets, respaldar archivos y desarrollar de forma formal.                                                                   |
| PUBLICACIÓN, MARKETING Y COMUNIDAD      | \$17.472.800 CLP  | Agrupar los costos necesarios para preparar el lanzamiento comercial: publicación en plataformas, trailer, press kit, redes sociales, publicidad y gestión de comunidad.                                                                             |
| OTROS / CONTINGENCIA                    | \$27.627.420 CLP  | Margen reservado para imprevistos, retrabajo, ajustes técnicos, cambios de alcance o variaciones de costos durante el desarrollo.                                                                                                                    |
| TOTAL PROYECTO IDEAL COMERCIAL          | \$211.810.220 CLP | Presupuesto estimado para transformar el MVP en una versión completa, pulida y publicable del videojuego.                                                                                                                                            |

**Tabla 18. Elaboración Propia.**

En síntesis, el mayor costo del proyecto ideal se concentra en el equipo profesional y el desarrollo técnico, ya que esta área sostiene la construcción completa del videojuego. Los demás bloques complementan esta base mediante producción audiovisual, herramientas de trabajo, preparación para publicación y un margen de contingencia necesario para enfrentar riesgos durante el desarrollo.

## Posibles fondos o vías de financiamiento

En caso de que el proyecto avance más allá del contexto académico, podrían evaluarse fondos o programas de apoyo orientados a emprendimiento, industrias creativas, innovación o internacionalización. Estos fondos no forman parte directa del MVP, pero sí podrían ser relevantes para una etapa posterior de crecimiento.

| FONDO O PROGRAMA                             | INSTITUCIÓN                                           | FUNDAMENTO DE PERTINENCIA                                                                                                                                                                                                                                                                                                                                                                | APORTE REFERENCIAL                |
|----------------------------------------------|-------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------|
| <b>FONDO AUDIOVISUAL / LÍNEA VIDEOJUEGOS</b> | Ministerio de las Culturas, las Artes y el Patrimonio | Squid Ink-Pulse podría calzar en este fondo porque corresponde a un videojuego con propuesta artística, narrativa y técnica propia. El proyecto no se limita a una demostración mecánica, sino que integra identidad visual, narrativa de crecimiento, diseño de personajes, ambientación submarina y una mecánica diferenciadora basada en riesgo-recompensa.                           | Entre \$25.000.000 y \$70.000.000 |
| <b>CAPITAL SEMILLA EMPRENDE</b>              | Sercotec                                              | El proyecto podría postular en una etapa inicial si el equipo decide formalizarlo como emprendimiento. Squid Ink-Pulse tiene potencial de transformarse en un producto comercial independiente, con un MVP validable, público objetivo definido y posibilidad de crecimiento mediante nuevas zonas, skins, mejoras y publicación digital.                                                | \$3.500.000                       |
| <b>SEMILLA INICIA</b>                        | Corfo                                                 | Squid Ink-Pulse podría ser pertinente porque posee una base de prototipo/MVP y requeriría validación técnica y comercial para avanzar hacia un producto real. Su propuesta presenta elementos de innovación dentro del género endless runner al incorporar una mecánica central basada en exposición controlada al peligro, lo que permite diferenciarlo frente a otros juegos casuales. | Hasta \$15.000.000                |
| <b>START-UP CHILE BUILD</b>                  | Start-Up Chile / Corfo                                | El proyecto podría calzar si se proyecta como una startup de entretenimiento digital o videojuego indie con potencial de escalamiento. Squid Ink-Pulse posee una idea validable, un MVP en desarrollo y posibilidades de expansión comercial mediante publicación en plataformas, contenido adicional y una identidad de marca propia.                                                   | \$15.000.000 equity-free          |

**Tabla 19. Elaboración Propia.**

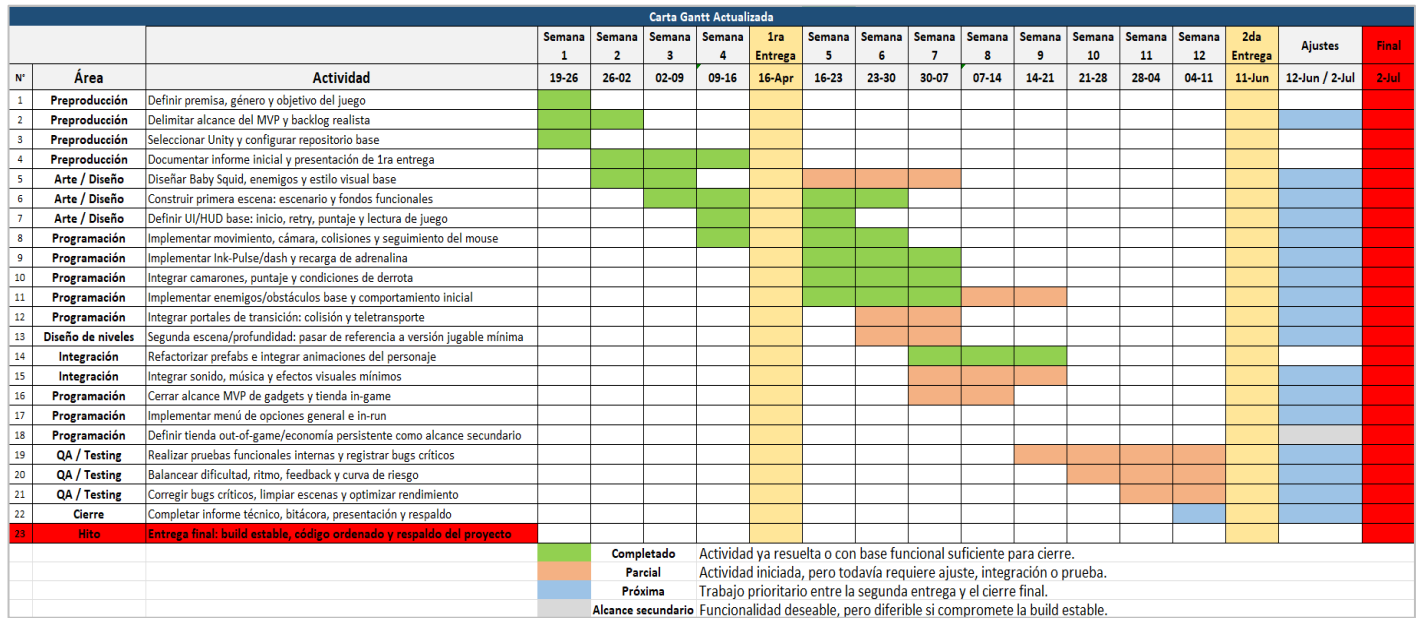
En síntesis, Squid Ink-Pulse podría optar a estos fondos porque combina tres dimensiones financiables: creación audiovisual interactiva, emprendimiento digital e innovación temprana. Su valor no está solo en ser un videojuego, sino en presentar una propuesta con identidad visual, narrativa, mecánica diferenciadora y proyección comercial.

## Metodología de trabajo y planificación

El desarrollo del proyecto se organiza bajo la metodología ágil SCRUM, estructurando el trabajo en sprints semanales con objetivos claros y entregables definidos. Cada sprint contempla planificación, ejecución y revisión, permitiendo una iteración constante del producto.

El equipo se organiza en roles funcionales, manteniendo comunicación continua para detectar problemas y ajustar el rumbo del desarrollo. Este enfoque facilita la adaptación a cambios, la priorización de tareas críticas y la entrega progresiva de funcionalidades, alineándose con un desarrollo incremental del videojuego.

### Carta Gantt - Actualizada



**Figura 19. Carta Gantt. Elaboración propia.**

La planificación general del proyecto se apoya en una Carta Gantt, que define las etapas principales del desarrollo y su distribución temporal. Esta herramienta permite visualizar:

- Fases del proyecto.
- Dependencias entre tareas.
- Plazos estimados de entrega.

La Carta Gantt funciona como una guía macro, mientras que SCRUM gestiona el trabajo a nivel micro (semanal), asegurando coherencia entre planificación a largo plazo y ejecución diaria.

Nuestra carta se divide en las siguientes fases:

### Hitos

- **1ra Entrega:** Concepto del juego 20/04 – **Cumplido.**
- **2da Entrega:** MVP Completo 11/06 – **Cumplido.**
- **3ra Entrega:** Build estable, código ordenado y respaldo del proyecto – **En proceso.**

## Áreas y detalle

- **Preproducción: Ya completa**
  - Definir premisa, género y objetivo del juego
  - Delimitar alcance del MVP y mecánicas prioritarias
  - Seleccionar motor de juego
  - Documentar en informe
- **Arte / Diseño: Ya completa**
  - Bocetar personaje principal y enemigos
  - Diseñar escenario y fondos base
  - Definir estilo visual de UI/HUD
- **Programación: Ya completa**
  - Implementar movimiento, cámara y colisiones
  - Implementar mecánica principal e interacción base
  - Implementar sistema de vida, daño y puntaje
  - Implementar enemigos
  - Integrar UI/HUD y flujo del juego
- **Diseño de niveles: Ya completa**
  - Construir nivel base
  - Montar nivel completo y progresión de juego
- **Integración: Ya completa**
  - Integrar animaciones de personaje y enemigos
  - Integrar sonido, música y efectos visuales
- **QA / Testing**
  - Realizar pruebas funcionales internas
  - Balancear dificultad, ritmo y feedback del jugador
  - Corregir bugs y optimizar rendimiento
- **Cierre**
  - Preparar demo para feria
  - Completar documentación técnica y presentación
  - Aplicar ajustes según retroalimentación

**\*Nota:** Las etapas de Arte / Diseño, Programación, Diseño de Niveles e Integración fueron reiterativas, de modo que se operaba sobre estas cíclicamente.

## Herramientas de trabajo

| HERRAMIENTA | FUNCIÓN                                                                                                                                                                                                                                                                                                                                                                                    |
|-------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| JIRA        | Plataforma de gestión de tareas alineada con SCRUM. Se emplea para organizar el flujo de trabajo en columnas (por hacer, en progreso, en revisión, hecho), asignar responsabilidades y hacer seguimiento del avance en cada sprint.                                                                                                                                                        |
| UNITY       | Motor de desarrollo utilizado para la implementación del videojuego. Permite integrar programación (C#), diseño de niveles, físicas, animaciones y UI en un entorno unificado, facilitando la creación del prototipo jugable.                                                                                                                                                              |
| GITHUB      | Utilizado como sistema de control de versiones, permitiendo gestionar el código fuente, mantener historial de cambios y facilitar el trabajo colaborativo mediante ramas (branches) y fusiones (merge). Garantiza trazabilidad y respaldo del desarrollo.                                                                                                                                  |
| CANVA       | Utilizado para organizar y facilitar el manejo y la alteración de sprites.                                                                                                                                                                                                                                                                                                                 |
| DISCORD     | Medio oficial de comunicación del equipo, en él se ejecutan las reuniones diarias de cumplimiento de tareas. Gracias a sus canales se puede organizar la información que se quiera comunicar. Además, poseemos el canal de voz “Programando” en caso de que alguien se encuentre trabajando el resto estemos al tanto, ayudando así a ejecutar tareas en paralelo si es que se requiriese. |

**Tabla 20. Tabla de Herramientas de Trabajo. Elaboración Propia.**

En conjunto, esta combinación de metodología y herramientas permite un desarrollo ordenado, iterativo y controlado, asegurando tanto la calidad del producto como el cumplimiento de los plazos establecidos.

## Sprints Ejecutados / Bitácora

### Sprint 1 — S1: Definiciones

| CATEGORÍA         | DETALLE                                                                                                                                          |
|-------------------|--------------------------------------------------------------------------------------------------------------------------------------------------|
| PERIODO           | 19-03-2026 al 26-03-2026                                                                                                                         |
| ESTADO            | Cerrado                                                                                                                                          |
| OBJETIVO          | Definir el proyecto y el MVP, dejando el juego conceptual y organizativamente listo para iniciar el prototipo.                                   |
| RESULTADO         | Objetivo cumplido: definición de MVP, gameplay loop, narrativa, identidad y configuración de herramientas como Jira, Gantt, repositorio y motor. |
| MÉTRICAS          | 9/9 issues funcionales completadas → 100% de cumplimiento real.                                                                                  |
| ACLARACIÓN        | 6 issues excluidas por corresponder a definición de roles, no a tareas funcionales del sprint.                                                   |
| STORY POINTS      | No registrados → no se puede medir velocidad ni precisión de planificación.                                                                      |
| TRABAJO REALIZADO | Base conceptual del MVP, loop, lore y brand; además de base operativa mediante herramientas, repositorio y motor.                                |
| OBSERVACIONES     | Roles modelados como issues; uso incompleto de métricas SCRUM.                                                                                   |
| RIESGOS           | Bajo impacto: desalineación metodológica, no técnica.                                                                                            |
| CONCLUSIÓN        | Sprint exitoso: proyecto definido y preparado para iniciar desarrollo.                                                                           |

**Tabla 21. Tabla de Sprint 1. Elaboración propia apoyada en Rovo, I.A. integrada en Jira.**

**Sprint 2 — S2: Base Visual y Técnica**

| CATEGORÍA     | DETALLE                                                                                                                                                                                                     |
|---------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| PERIODO       | 27-03-2026 al 09-04-2026                                                                                                                                                                                    |
| ESTADO        | Cerrado                                                                                                                                                                                                     |
| OBJETIVO      | Definir el estilo visual del juego y construir la infraestructura mínima de un primer prototipo jugable.                                                                                                    |
| RESULTADO     | Objetivo mayoritariamente cumplido: base visual consolidada, incluyendo identidad, personaje, UI, escenarios y enemigos; además de una base técnica funcional en Unity, mecánicas base, menú y repositorio. |
| MÉTRICAS      | 11/13 issues completadas → ≈85% de cumplimiento.                                                                                                                                                            |
| ACLARACIÓN    | Issues de rol no consideradas como tareas; 2 issues quedaron en proceso y continúan en S3.                                                                                                                  |
| PENDIENTES    | Documentación técnica y testing del prototipo.                                                                                                                                                              |
| OBSERVACIONES | Ausencia de story points; avance desbalanceado hacia desarrollo por sobre QA y documentación.                                                                                                               |
| RIESGOS       | Deuda de documentación y validación → posible retrabajo o inconsistencias.                                                                                                                                  |
| CONCLUSIÓN    | Sprint exitoso en términos de prototipo y base visual; requiere fortalecer documentación y testing en siguientes iteraciones.                                                                               |

**Tabla 22. Tabla de Sprint 2. Elaboración propia apoyada en Rovo, I.A. integrada en Jira.**



### Sprint 3 — S3: Prototipo Jugable

| CATEGORÍA           | DETALLE                                                                                                                                                     |
|---------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------|
| PERIODO             | 09-04-2026 al 16-04-2026                                                                                                                                    |
| ESTADO              | Activo, no cerrado en Jira                                                                                                                                  |
| OBJETIVO            | Construir un prototipo jugable mínimo con el ciclo base: jugar → enfrentar amenaza → perder o sobrevivir → reintentar.                                      |
| RESULTADO           | Objetivo parcialmente logrado: avances en UI, menú de pausa, identidad y definiciones; sin embargo, el loop jugable aún no está completamente implementado. |
| MÉTRICAS            | 5/14 issues en “Hecho” → ≈36% de cumplimiento, más 1 issue en revisión.                                                                                     |
| AVANCES CLAVE       | Menú de pausa, diseño e implementación, definición de enemigos, presentación inicial y logo.                                                                |
| PENDIENTES CRÍTICOS | Mecánicas core del gameplay, enemigos, boss, menú de opciones y cierre del loop jugable.                                                                    |
| ARRASTRE            | Documentación y testing continúan desde S2.                                                                                                                 |
| OBSERVACIONES       | Progreso centrado en UI y diseño; desarrollo del core jugable rezagado.                                                                                     |
| RIESGOS             | Prototipo incompleto, deuda de QA y documentación, posible desalineación del diseño.                                                                        |
| CONCLUSIÓN          | Sprint en estado incompleto; requiere priorizar mecánicas centrales y cierre del loop jugable para cumplir el objetivo.                                     |

**Tabla 23. Tabla de Sprint 3. Elaboración propia apoyada en RoVo, I.A. integrada en Jira.**

## Sprint 4 — S4: Mecánicas clave MVP

| CATEGORÍA     | DETALLE                                                                                                                                                                                                                                                                                                 |
|---------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| PERIODO       | 07-05-2026 al 28-05-2026                                                                                                                                                                                                                                                                                |
| ESTADO        | Finalizado, con 1 issue pendiente                                                                                                                                                                                                                                                                       |
| OBJETIVO      | Cumplir las mecánicas comprometidas en el MVP presentado, priorizando la implementación de sistemas jugables centrales.                                                                                                                                                                                 |
| RESULTADO     | Objetivo mayoritariamente cumplido: se implementaron y refinaron mecánicas clave del MVP, incluyendo enemigos, jefe, ataque tipo “pared”, pez dealer y menú de tienda. Quedó pendiente la implementación general del menú de opciones.                                                                  |
| MÉTRICAS      | 8/9 issues funcionales completadas → ≈89% de cumplimiento real.                                                                                                                                                                                                                                         |
| ACLARACIÓN    | Se excluyen 6 issues constantes correspondientes a roles del equipo, ya que no representan tareas funcionales del sprint.                                                                                                                                                                               |
| AVANCES CLAVE | Implementación de mecánica de enemigos, mecánica de jefe con alejamiento de cámara, ataque tipo “pared” mediante el jefe, refinamiento del enemigo especial S.S. Carnage, implementación del pez dealer, menú de tienda, diseño del menú de opciones y evaluación de implementaciones mediante testing. |
| PENDIENTES    | Implementación de Menú de Opciones general.                                                                                                                                                                                                                                                             |
| OBSERVACIONES | El sprint se concentró principalmente en features asociadas al MVP, con 5 de 9 issues funcionales orientadas a mecánicas. También se incorporaron tareas de diseño y testing, lo que permitió validar parcialmente las implementaciones realizadas.                                                     |
| RIESGOS       | Persistencia de una deuda funcional menor asociada al menú de opciones. Riesgo moderado de integración si esta funcionalidad se posterga demasiado respecto del cierre del MVP.                                                                                                                         |
| CONCLUSIÓN    | Sprint exitoso en términos de avance técnico: se completaron las mecánicas centrales comprometidas para el MVP y se fortaleció el núcleo jugable. Se requiere cerrar el menú de opciones para consolidar el flujo completo del producto.                                                                |

**Tabla 24. Tabla de Sprint 4. Elaboración propia apoyada en Rovo, I.A. integrada en Jira.**

**Sprint 5 — S5: Tienda y Portales**

| CATEGORÍA     | DETALLE                                                                                                                                                                                                                                                                                                 |
|---------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| PERIODO       | 28-05-2026 al 05-06-2026                                                                                                                                                                                                                                                                                |
| ESTADO        | Finalizado, con tareas pendientes y en proceso                                                                                                                                                                                                                                                          |
| OBJETIVO      | Implementar portales para transición de escenarios y una tienda funcional basada en la economía de camarones.                                                                                                                                                                                           |
| RESULTADO     | Objetivo mayoritariamente cumplido. Se implementaron portales, tienda, pez dealer, persistencia de camarones, inventario, gadgets, menú de Game Over y correcciones funcionales. Quedaron pendientes ajustes de opciones, parámetros de dificultad y cierre de elementos asociados al spawn y tutorial. |
| MÉTRICAS      | 18 de 22 tareas completadas, equivalente a un 82% de cumplimiento cerrado.                                                                                                                                                                                                                              |
| ACLARACIÓN    | No se consideran las tareas constantes asociadas a roles, ya que no corresponden a entregables funcionales del sprint.                                                                                                                                                                                  |
| AVANCES CLAVE | Se consolidaron sistemas centrales del MVP: economía, tienda, gadgets, inventario, portales y corrección de errores de integración.                                                                                                                                                                     |
| PENDIENTES    | Menú de opciones, parámetros de dificultad, algoritmo de spawn y nivel tutorial.                                                                                                                                                                                                                        |
| OBSERVACIONES | El sprint tuvo una alta carga técnica y permitió integrar sistemas relevantes para completar el flujo jugable. La mayor parte del trabajo se concentró en programación, integración y corrección de errores.                                                                                            |
| RIESGOS       | Persisten riesgos moderados asociados al balance del juego y a la experiencia inicial del jugador, especialmente por el tutorial y el sistema de spawn.                                                                                                                                                 |
| CONCLUSIÓN    | Sprint exitoso en términos funcionales, ya que permitió consolidar la tienda, economía, portales, gadgets e inventario. Para cerrar el flujo del MVP, se requiere finalizar los ajustes de dificultad, spawn y tutorial.                                                                                |

**Tabla 25. Tabla de Sprint 5. Elaboración propia apoyada en Rovo, I.A. integrada en Jira.**

## Sprint 6 — S6: Presentación y segunda escena

| CATEGORÍA     | DETALLE                                                                                                                                                                                                                                                                                                                                                                                               |
|---------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| PERIODO       | 05-06-2026 al 13-06-2026                                                                                                                                                                                                                                                                                                                                                                              |
| ESTADO        | Finalizado, con tareas pendientes y en proceso                                                                                                                                                                                                                                                                                                                                                        |
| OBJETIVO      | Ajustar el flujo del juego para la presentación, incorporando una segunda escena referencial, mejoras visuales y correcciones funcionales.                                                                                                                                                                                                                                                            |
| RESULTADO     | Objetivo parcialmente cumplido. Se consolidaron elementos relevantes para la presentación, como portales, diseño del segundo escenario, correcciones de transición, refactorización del personaje principal como prefab, presentación en Canva y animaciones del S.S. Carnage e Ink-Pulse. Quedaron pendientes sistemas complementarios asociados a menús, tienda externa y parámetros de dificultad. |
| MÉTRICAS      | 13 de 20 tareas completadas, equivalente a un 65% de cumplimiento cerrado.                                                                                                                                                                                                                                                                                                                            |
| ACLARACIÓN    | No se consideran las tareas constantes asociadas a roles, ya que no corresponden a entregables funcionales del sprint.                                                                                                                                                                                                                                                                                |
| AVANCES CLAVE | Se avanzó en la integración de portales, segunda escena, corrección de errores de transición, material visual para presentación, animaciones y diseños complementarios.                                                                                                                                                                                                                               |
| PENDIENTES    | Menú de opciones, menú de opciones durante la partida, tienda out of game, parámetros de dificultad, algoritmo de spawn, nivel tutorial e Informe N°2.                                                                                                                                                                                                                                                |
| OBSERVACIONES | El sprint estuvo orientado principalmente a preparar una versión presentable del proyecto, priorizando integración visual, correcciones funcionales y material expositivo. La carga de trabajo se concentró especialmente en programación, diseño e integración.                                                                                                                                      |
| RIESGOS       | Persisten riesgos asociados al cierre del flujo completo del MVP, especialmente por sistemas aún incompletos vinculados a menús, tienda externa, spawn y tutorial.                                                                                                                                                                                                                                    |
| CONCLUSIÓN    | Sprint parcialmente exitoso: permitió consolidar una base funcional y visual adecuada para la presentación. Sin embargo, el MVP aún requiere cerrar sistemas pendientes para alcanzar una build más estable y completa.                                                                                                                                                                               |

**Tabla 26. Tabla de Sprint 6. Elaboración propia apoyada en Roovo, I.A. integrada en Jira.**

## Sprint 7 — S7: Últimas implementaciones

| CATEGORÍA             | DETALLE                                                                                                                                                                                                                                                                                                             |
|-----------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| PERIODO               | 13-06-2026 al 18-06-2026                                                                                                                                                                                                                                                                                            |
| ESTADO                | Activo                                                                                                                                                                                                                                                                                                              |
| OBJETIVO              | Implementar funcionalidades finales del MVP, enfocadas en persistencia de datos, tienda out of game, sistema de skills, skins y almacenamiento mediante archivos JSON.                                                                                                                                              |
| RESULTADO             | Avance parcial. Se completó el algoritmo de spawn, fortaleciendo la generación de amenazas y el flujo jugable. Además, se mantienen en proceso tareas relevantes como el nivel tutorial, el Informe N°2, la definición de parámetros de dificultad y la tienda out of game. Persisten pendientes asociados a menús. |
| MÉTRICAS              | 1 de 7 tareas completadas, equivalente a un 14% de cumplimiento cerrado. Considerando tareas en proceso, el avance iniciado alcanza 5 de 7 tareas, equivalente a un 71%.                                                                                                                                            |
| ACLARACIÓN            | No se consideran las tareas constantes asociadas a roles, ya que no corresponden a entregables funcionales del sprint.                                                                                                                                                                                              |
| AVANCES CLAVE         | Finalización del algoritmo de spawn, avance en tutorial, documentación del informe, parámetros de dificultad y tienda out of game.                                                                                                                                                                                  |
| PENDIENTES            | Menú de opciones, menú de opciones durante la partida, cierre del tutorial, tienda out of game, parámetros de dificultad e Informe N°2.                                                                                                                                                                             |
| OBSERVACIONES         | El sprint concentra tareas finales necesarias para consolidar el MVP y preparar una build más completa. Aún existen tareas críticas en proceso, por lo que el cierre del sprint requiere priorización estricta.                                                                                                     |
| RIESGOS               | Riesgo alto asociado al tutorial, por su impacto directo en la experiencia inicial del jugador. También persiste riesgo por funcionalidades de menú y tienda externa aún no cerradas.                                                                                                                               |
| CONCLUSIÓN PRELIMINAR | Sprint en desarrollo, con un avance técnico relevante tras completar el algoritmo de spawn. Para cerrar adecuadamente el MVP, se requiere priorizar tutorial, menús, tienda externa y parámetros de dificultad.                                                                                                     |

**Tabla 27. Tabla de Sprint 7. Elaboración propia apoyada en RoVo, I.A. integrada en Jira.**

## Conclusiones

### Alcance actual del proyecto

Squid Ink-Pulse ha avanzado desde una propuesta conceptual hacia un MVP funcional en desarrollo. El proyecto ya permite validar su núcleo jugable: movimiento continuo, evasión de amenazas, carga del Ink-Pulse mediante proximidad al peligro y uso estratégico de este recurso en momentos críticos.

### Viabilidad del proyecto

El proyecto se considera viable dentro del contexto académico, principalmente porque mantiene un alcance controlado y coherente con el tiempo disponible. La elección de un endless runner 2D permite concentrar el esfuerzo en mecánicas centrales, balance, interfaz y progresión, sin ampliar innecesariamente la complejidad del desarrollo.

No obstante, la viabilidad final depende de cerrar pendientes específicos: persistencia completa, tienda global, tutorial, menú de opciones, balance de dificultad, QA formal y pulido audiovisual. Estos elementos no modifican la identidad del juego, pero sí son necesarios para consolidar una build estable.

### Avance e implementación

El avance del proyecto es significativo. Se han implementado sistemas relevantes como movimiento del jugador, Ink-Pulse, graze zone, enemigos, camarones, gadgets, tienda in-run, portales, HUD, pausa, Game Over y evento de boss.

El logro más importante corresponde a la implementación del Ink-Pulse y su relación con la mecánica de riesgo-recompensa, ya que este sistema representa la identidad principal del videojuego. Aun así, el proyecto requiere una etapa final de integración, pruebas y ajustes para asegurar que todos los sistemas funcionen de forma equilibrada y comprensible para el jugador.

### Aporte de SCRUM al desarrollo

La metodología SCRUM permitió organizar el trabajo en sprints, definir prioridades y controlar el avance del equipo de forma progresiva. Gracias a esta estructura, el proyecto pudo avanzar por etapas: primero la definición conceptual, luego la base visual y técnica, posteriormente las mecánicas centrales, y finalmente los ajustes, documentación y cierre del MVP.

El uso de Jira, GitHub y Discord facilitó la asignación de tareas, el seguimiento del progreso, la comunicación interna y el control de versiones. Esto permitió detectar pendientes, reorganizar prioridades y mantener una trazabilidad clara del desarrollo.

### Cierre general

En conclusión, Squid Ink-Pulse es una propuesta viable, coherente y con un avance técnico relevante. El proyecto logró transformar una idea inicial en una base jugable funcional, manteniendo una identidad clara basada en riesgo, reacción y progresión. Aunque aún requiere ajustes finales, el trabajo desarrollado demuestra una base sólida para completar una versión estable y presentable del MVP.

## Referencias y Bibliografía

Adams, E. (2014). *Fundamentals of game design* (3rd ed.). New Riders.

Computrabajo. (s. f.). *Salarios: Dibujante*. Recuperado el 14 de junio de 2026, de <https://cl.computrabajo.com/salarios/dibujante>

Computrabajo. (s. f.). *Salarios: Diseñadores/as gráficos*. Recuperado el 14 de junio de 2026, de <https://cl.computrabajo.com/salarios/disenadoresas-graficos>

Computrabajo. (s. f.). *Salarios: Tester QA*. Recuperado el 14 de junio de 2026, de <https://cl.computrabajo.com/salarios/tester-qa>

Corporación de Fomento de la Producción. (s. f.). *Semilla Inicia*. Corfo. Recuperado el 14 de junio de 2026, de [https://www.corfo.gob.cl/sites/cpp/convocatoria/semilla\\_inicia/](https://www.corfo.gob.cl/sites/cpp/convocatoria/semilla_inicia/)

Duoc UC. (s. f.). *Cuánto gana un programador en Chile*. Recuperado el 14 de junio de 2026, de [https://www.duoc.cl/?noticia\\_post\\_type=cuanto-gana-un-programador-en-chile](https://www.duoc.cl/?noticia_post_type=cuanto-gana-un-programador-en-chile)

Fox, T. (2018). *Deltarune* [Videojuego]. Toby Fox.

Kiloo & SYBO Games. (2012). *Subway Surfers* [Videojuego]. Kiloo.

McMillen, E., & Nicalis, Inc. (2011). *The Binding of Isaac* [Videojuego]. Nicalis.

Ministerio de las Culturas, las Artes y el Patrimonio. (2024). *Bases concurso público: Línea videojuegos, convocatoria 2025* [PDF]. Fondo Audiovisual. <https://www.fondosdecultura.cl/wp-content/uploads/2024/07/6-FA-VIDEOJUEGOS-2025.pdf>

Nguyen, D. (2013). *Flappy Bird* [Videojuego]. .GEARS Studios.

O'Dor, R. K., Boucher-Rodoni, R., & Wells, M. J. (Eds.). (2002). *The biology of squid*. Smithsonian Institution Press.

Schwaber, K. (2004). *Agile project management with Scrum*. Microsoft Press.

Servicio de Cooperación Técnica. (s. f.). *Capital Semilla Emprende*. Sercotec. Recuperado el 14 de junio de 2026, de <https://www.sercotec.cl/programas/capital-semilla-emprende/>

Start-Up Chile. (s. f.). *Build*. Recuperado el 14 de junio de 2026, de <https://startupchile.org/en/apply/build/>

Sutherland, J. (2014). *Scrum: The art of doing twice the work in half the time*. Crown Business.

Unity Technologies. (2024). *Unity manual*. Unity Documentation. <https://docs.unity3d.com/Manual/index.html>

WageIndicator Foundation. (s. f.). *Función y salario: Músicos, cantantes y compositores*. TuSalario.org. Recuperado el 14 de junio de 2026, de <https://wageindicator.org/es-cl/trabajo-en-chile/funcion-y-salario/musicos-cantantes-y-compositores>